

Universal, Hard-Routed and Soft Mixture-of-Experts Image Restoration, and Style-Conditioned Face-to-Sketch Generation

Mustafa Faraz

Department of Computer Science

National University of Computer and Emerging Sciences (FAST-NUCES), Islamabad

mustafafaraz87@gmail.com

Abstract—This report presents four related generative systems built on a single data pipeline. Clean images of the Oxford-IIIT Pet dataset are corrupted at load time by one of three degradations—salt-and-pepper noise, Gaussian blur or rectangular occlusion—or left clean, and three systems of increasing routing sophistication learn to invert them. The first is a universal convolutional denoising autoencoder with a genuine compressed bottleneck, which is never told which corruption it is facing. The second replaces one shared model by a four-class corruption classifier that routes each image to one of three specialist autoencoders, with an identity bypass for clean inputs. The third makes that routing differentiable: a temperature-scaled softmax gate, initialised from the classifier, assigns continuous weights to the identity branch and the three specialists, and the whole system is fine-tuned jointly under a loss that combines reconstruction, structural similarity, corruption classification and a routing-balance regulariser. The fourth system is a conditional GAN that translates a facial photograph into a pencil sketch in one of three artist styles of the FS2K dataset, using a U-Net generator whose decoder is modulated by a learned style embedding and a PatchGAN discriminator conditioned by projection. Every model is tuned with Optuna, tracked with Weights & Biases, exported to ONNX with a verified parity check, and served through a FastAPI backend behind a React interface that runs under Docker Compose. [TODO: headline Task 1 result on the 33,021 corrupted test entries (deployed variant) and the skip-ablation finding] [TODO: one sentence of headline results for the other systems once the Kaggle runs finish: Task 2 classifier macro-F1 and routing gain, Task 3 soft-routing gain over hard routing, Task 4 per-style SSIM and L_1]

Index Terms—image restoration, denoising autoencoder, mixture of experts, conditional generative adversarial networks, hyperparameter optimisation, ONNX

I. INTRODUCTION

Image restoration systems are usually built one degradation at a time. A denoiser is trained on noise, a deblurring network on blur, an inpainting network on holes, and the choice between them is left to whoever runs the pipeline. That arrangement is convenient for research but awkward in deployment, where the degradation afflicting an incoming image is rarely announced. Two families of answers have emerged. One trains a single model across degradations and relies on its capacity to cover all of them [1], [2], [3]; the other keeps specialised components and adds a mechanism that decides which to use [4], [5].

This assignment asks for both, and for the step that connects them. Using one fixed data pipeline we build three restoration systems that differ only in how the knowledge of the corruption enters the computation.

- **Task 1** is a universal denoising autoencoder. It receives a corrupted image and nothing else: no label, no severity, no mask. All four input conditions—clean, salt-and-pepper, blur, occlusion—must be handled by one encoder, one compressed latent representation and one decoder.
- **Task 2** makes the corruption explicit. A convolutional classifier predicts which of the four conditions holds, and that discrete decision selects one of three specialist autoencoders, or an identity bypass when the image is judged clean. The system is evaluated twice: with the true label from the test manifest (*oracle routing*, which isolates how good the specialists are) and with the classifier’s own prediction (*predicted routing*, which is the operational system).
- **Task 3** replaces the $\arg\max$ with a temperature-scaled softmax. The gate is the Task 2 classifier, the experts are the Task 2 specialists, and the output is their weighted sum. Because that sum is differentiable, the gate and the experts can be trained jointly on the final reconstruction error, with a balance regulariser to keep the gate from collapsing onto a single branch.

Task 4 is a different problem on a different dataset: paired image-to-image translation with a categorical condition. Given a facial photograph and one of three sketch styles, a conditional GAN must produce the sketch that the corresponding artist would have drawn.

Three themes run through the report. The first is *evidence*: every architectural choice—where the bottleneck sits, whether skip connections are allowed, which loss, which upsampling operator, which balance regulariser, how the style condition is injected—is argued from published results and, where the literature is ambiguous, measured. The second is *honesty about what the numbers mean*: PSNR is capped for identical images, clean inputs through an identity bypass are exact rather than merely excellent, FID is biased at small sample sizes, and the FS2K test set contains only 46 images of one style. Each of these is stated where the corresponding result is

presented rather than buried. The third is *deployability*: all seven inference models are exported to ONNX and checked against their PyTorch originals, and the complete application starts with a single Docker Compose command.

The implementation, including the data preparation, training, search, evaluation and export scripts, the backend and the frontend, is at <https://github.com/fsdev87/generative-ai-A1>. A demonstration video is at [TODO: YouTube link to the 5–7 minute demonstration video].

II. RELATED WORK

A. Denoising autoencoders and restoration networks

The denoising autoencoder was introduced as a representation learner: an autoencoder trained to reconstruct a clean input from a stochastically corrupted one cannot settle for the identity and must capture the structure of the data [6], [7]. The corruption is applied during training only, which is exactly the runtime scheme used here. The compressed central layer is what makes the representation compact [8], and convolutional encoder-decoder stacks make the idea practical on images [9], [10]. DnCNN showed that one network can cover a *range* of noise levels rather than one model per level [1], the precedent for a single model covering several corruption families.

Two architectural traditions meet in Task 1. U-Net concatenates every encoder stage into the mirrored decoder stage [11], and RED-Net adds symmetric skip connections to a very deep restoration network, reporting both faster convergence through better gradient flow and better detail recovery because the skips “pass image details from convolutional layers to de-convolutional layers” [12]. That second mechanism is precisely what a bottleneck is supposed to forbid: a full-resolution skip lets the decoder reconstruct detail the latent never encoded. Context Encoders take the opposite route for inpainting, using a narrow bottleneck and no skips, because occluded content must be synthesised from context rather than copied [13]. Section IV quantifies both sides of this trade-off for the bottleneck model rather than assuming them.

B. Routing and all-in-one restoration

RL-Restore assembles a toolbox of small, differently specialised CNNs and learns by reinforcement learning which tools to apply and in what order [4]; Path-Restore extends the idea to spatially varying routing, selecting a path per image region with a difficulty-regulated reward [5]. Both report that a set of small specialists plus a routing policy is more parameter-efficient than one large network—the hypothesis Task 2 tests in its simplest form, with a single image-level decision and a supervised classifier instead of a learned policy. The opposite design removes the router entirely: AirNet infers the degradation with a contrastive encoder that then guides restoration [2], and PromptIR encodes degradation-specific information in lightweight learned prompts that modulate a single network [3]. These all-in-one models are the state of the art for the setting of Task 1; they are trained on far more data with far larger backbones, and no comparison with them is claimed here.

C. Mixtures of experts

The mixture of experts originates with Jacobs et al. [14], who distinguish a *cooperative* error function, in which the target is compared with a blend $\sum_i p_i o_i$ of expert outputs, from a *competitive* one, $\sum_i p_i \|d - o_i\|^2$, in which each expert must produce the whole output. They argue that the cooperative form “does not encourage localization”, because each expert only has to cancel the residual left by the others, and that it “tends to lead to solutions in which many experts are used for each case”. Jordan and Jacobs later gave the construction its statistical form as a mixture model trained by EM [15]; Yuksel et al. survey two decades of variants [16].

The modern, large-scale form adds sparsity. Shazeer et al. introduce noisy top- k gating and observe the central pathology: “the gating network tends to converge to a state where it always produces large weights for the same few experts”, an imbalance that is self-reinforcing because favoured experts are trained faster and therefore chosen more often [17]. Their remedy is a pair of soft constraints on batch statistics, discussed quantitatively in Section VI. Switch Transformers simplify the two losses into one [18], [19], and Soft MoE removes the problem structurally: every expert slot receives a weighted average of all tokens, so the model is “immune to token dropping and expert unbalance” and needs no auxiliary balance loss at all [20]. The system in Task 3 is dense and soft in the same sense—all four branches always run—but small enough that the gate, not the capacity, is the object of study. The wider framing is conditional computation, in which each input traverses only part of the network [21], [22]; Eigen et al. already observed the gating-collapse failure mode and addressed it with a hard constraint early in training, and Bengio et al. with a soft constraint on the batchwise average of each gate—the family to which the regulariser used here belongs.

D. Conditional image-to-image translation

Conditional GANs supply the condition to both generator and discriminator [23], [24]. pix2pix is the canonical paired translation model: a U-Net generator, a 70×70 PatchGAN discriminator, and an L_1 term with weight $\lambda = 100$ added to the adversarial loss, L_1 rather than L_2 because it “encourages less blurring” [25]. Isola et al. also report that removing the conditioning from the discriminator makes the generator collapse to nearly the same output regardless of input, which is direct evidence for conditioning both networks.

How a *categorical* condition should enter is a design question with several published answers. The simplest spatially replicates a label map and concatenates it to the input [26]. Conditional instance normalisation learns a per-condition scale and shift, letting a single network render many styles [27], a mechanism generalised as FiLM [28], [29] and applied to language-driven vision [30]; AdaIN extends it to arbitrary, unseen styles [31]. On the discriminator side, the auxiliary classifier of AC-GAN [32] competes with the projection discriminator, which adds an inner product between the embedded condition and the discriminator’s feature vector and

is motivated by the role of the condition in the underlying probabilistic model [33]. Section VII explains, and measures, why this work conditions the generator by FiLM and the discriminator by projection.

Face photo-to-sketch synthesis itself has a long history, from multiscale Markov random fields on the CUHK datasets [34] to the FS2K benchmark used here [35].

III. DATASET PREPARATION

Tasks 1–3 use the Oxford-IIIT Pet dataset [36], [37]: 7,349 photographs of 37 cat and dog breeds. Task 4 uses FS2K [35], [38], 2,104 photograph-sketch pairs in three artist styles. All data preparation is implemented in `src/data/` and the resulting splits and manifests are committed to the repository, so every run of every task sees identical data.

A. Splits

The official `trainval` list of the Oxford-IIIT Pet dataset contains 3,680 images and the official `test` list 3,669. As the brief requires, the development portion is divided 80/20 with random seed 42, giving **2,944 training** and **736 validation** images; the official test set of **3,669** images is left untouched until the final evaluation. All three splits are disjoint and all 37 breeds appear in each: the training split holds 71–91 images per breed (953 cats, 1,991 dogs), the validation split 9–29 per breed (235 cats, 501 dogs) and the test split 88–100 per breed (1,183 cats, 2,486 dogs). Breed labels are not used by any model—the clean image is the reconstruction target—but the per-breed counts confirm that the random split did not concentrate any breed in one partition.

Every image is converted to RGB and resized to 128×128 with bicubic resampling and no crop, then cached once as a `uint8` array so that training never re-decodes a JPEG. Pixel values are scaled to $[0,1]$ and stored as `float32` in NCHW order; all restoration models end in a sigmoid.

B. Corruption configuration

Four input conditions are used, in the fixed order (`clean`, `salt`, `blur`, `occlusion`) that indexes the classifier logits, the mixture weights and every table in this report; index 0 is always the clean or identity branch. Table I gives the training ranges, from which severities are sampled uniformly at training time, and the three fixed severity levels used for testing.

Three details of the implementation (`src/data/corruptions.py`) matter for the results.

Salt-and-pepper noise selects each pixel independently with probability p and sets all three channels to 0 or 1 with equal probability, so the corruption is a pure impulse: a hit pixel carries no information about its clean value.

Gaussian blur is separable with reflect padding and matches `torchvision's gaussian_blur` to within 10^{-5} , verified by a unit test. The kernel is truncated to k taps, and this truncation has a consequence that was discovered during data

preparation and changed how severity is labelled. The standard deviation of the kernel that is actually applied,

$$s(k, \sigma) = \left(\sum_{i=0}^{k-1} g_i \left(i - \frac{k-1}{2} \right)^2 \right)^{1/2}, \quad (1)$$

in which g denotes the normalised k -tap Gaussian kernel, saturates as σ grows, because a wide Gaussian truncated to k taps flattens towards a box filter whose standard deviation is bounded by $\sqrt{(k^2 - 1)/12}$. At $k = 3$ that bound is 0.816 px, so the configuration (3, 2.5) has strength 0.805 px—*milder* than the medium test level (5, 1.5) at 1.195 px, although its σ is larger. Ranking blur severity by σ alone is therefore wrong. Severity labels are instead assigned by binning $s(k, \sigma)$ with edges at the midpoints between the three test levels, 0.921 and 1.480 px. Regenerating the validation manifest under this rule left every corruption setting bit-identical and corrected **86 of its 184** blur severity labels.

Occlusion places n non-overlapping black rectangles whose union covers the target fraction to within one percentage point. The target area is divided among the rectangles by a Dirichlet draw, each rectangle receives a random aspect ratio in $[1/2, 2]$, and positions are rejected until no two overlap; the achieved coverage is recomputed from the union mask and resampled if it leaves the required range, so the stored coverage is the true one rather than the request.

A consequence of these definitions is worth stating before any result is reported: **all three test “high” levels sit at the upper edge of the training ranges**. Salt-and-pepper high uses $p = 0.15$, exactly the top of the training range; occlusion high covers 35%, exactly the top; and blur high, (7, 2.5) at strength 1.765 px, is the single strongest blur the training distribution can produce, reachable only at $k = 7$ with σ near its maximum. Under uniform sampling of (k, σ) only about 15% of training blurs fall in the “high” band at all, and none exceeds the test level. The high severities are thus the sparsest part of the training distribution, and the weakest results are expected there—not because the models fail, but because the brief’s test protocol probes the edge of what training covers.

C. Runtime corruption and deterministic manifests

Training corruptions are generated *at load time*: each access to a training image draws a fresh condition and a fresh severity, applies the corruption in NumPy and returns the pair (corrupted input, clean target) together with the integer label that produced it. Nothing corrupted is ever written to disk, so an image is seen under a different degradation in every epoch, which acts as a strong data augmentation and provides the classifier of Task 2 with free, perfectly accurate labels. A random horizontal flip is applied to the clean image before corruption.

Validation and test corruptions must instead be identical across runs, models and machines. They are therefore generated once into JSON *manifests* that store, for each entry, the corruption type, the severity level, every parameter (p ; k and σ ; the number of rectangles, achieved coverage and exact rectangle coordinates) and the random seed. Applying a stored

TABLE I

CORRUPTION CONFIGURATION. TRAINING SEVERITIES ARE SAMPLED FROM THE RANGES ON EVERY IMAGE LOAD; THE TEST SEVERITIES ARE FIXED BY THE BRIEF AND STORED IN THE TEST MANIFEST. BLUR STRENGTH IS THE STANDARD DEVIATION IN PIXELS OF THE KERNEL THAT IS ACTUALLY APPLIED AFTER TRUNCATION TO k TAPS.

Condition	Training range (sampled per load)	Test levels (low / medium / high)	Test strength
Clean	image passed through unchanged	—	—
Salt-and-pepper	$p \sim \mathcal{U}(0.02, 0.15)$; hit pixels set to 0 or 1 equiprobably	$p = 0.03 / 0.08 / 0.15$	—
Gaussian blur	$k \in \{3, 5, 7\}$ uniformly, $\sigma \sim \mathcal{U}(0.5, 2.5)$	$(3, 0.7) / (5, 1.5) / (7, 2.5)$	0.647 / 1.195 / 1.765 px
Rectangular occlusion	$n \in \{1, 2, 3\}$ rectangles covering $\mathcal{U}(0.10, 0.35)$ of the area	$n = 1, 2, 3$ at 10% / 20% / 35%	—

TABLE II

VALIDATION MANIFEST (736 ENTRIES, SEED 1001): ONE DETERMINISTIC CORRUPTION PER VALIDATION IMAGE, BALANCED ACROSS THE FOUR CONDITIONS.

Condition	Low	Medium	High	Total
Clean	(no severity)			184
Salt-and-pepper	46	97	41	184
Gaussian blur	92	62	30	184
Rectangular occlusion	36	91	57	184
Total				736

specification is fully deterministic, so the manifests reproduce the corrupted images exactly without storing them.

The **validation manifest** (seed 1001) assigns *one* condition to each of the 736 validation images, balanced across the four classes: exactly 184 entries each of clean, salt-and-pepper, blur and occlusion. Severities are drawn from the training ranges, so the validation distribution matches training. Table II gives the resulting severity composition. The distribution across levels is not uniform, and for a well understood reason: severity is binned on a physical quantity (p , covered fraction, blur strength) whose uniform sampling does not produce uniform bins. For blur the effect is strongest, since $k = 3$ can never reach the medium band at all.

The **test manifest** (seed 2002) is exhaustive instead of sampled: every one of the 3,669 test images appears in ten variants—clean, plus each of the three corruptions at each of the three fixed severity levels—giving **36,690 entries**, exactly 3,669 per (type, level) group. Because every image appears under every condition, comparisons across corruptions and severities are paired. The test set is balanced per (type, level) group but not per class: the clean class has 3,669 entries and each corruption 11,007, so for the Task 2 classifier macro-averaged metrics weight the clean class as much as each corruption, whereas accuracy (which equals micro-averaged F1 for single-label classification) is dominated by the corruptions. The achieved occlusion coverage across the test manifest is tightly controlled: mean 0.1000, 0.2000 and 0.3500 for the three levels, with extremes of 0.0987–0.1016, 0.1972–0.2028 and 0.3462–0.3544.

Figure 1 shows the four conditions at the three fixed test severities.

D. Class-balanced sampling

Wherever batches must be balanced—the Task 2 classifier and the Task 3 mixture—a custom batch sampler emits batches of (index, condition) pairs containing exactly $B/4$ images of each condition, reshuffling the images every epoch and dropping the final incomplete batch; batch sizes are therefore multiples of four. Because the condition is drawn by the sampler rather than by the dataset, perfect balance costs nothing: there is no minority class to oversample. This is the strongest form of the remedy that Buda et al. find dominant for class imbalance in CNNs, namely oversampling to the point where the imbalance is eliminated entirely [39]. It also has a second consequence that Section VI depends on: in an exactly balanced batch, a perfectly routing gate assigns mean weight exactly $1/4$ to each branch, so the routing-balance regulariser is zero at the optimum of the classification objective instead of fighting it.

E. FS2K preparation for Task 4

FS2K provides 2,104 photograph-sketch pairs with an official division into 1,058 training and 1,046 test pairs and a style label in $\{0, 1, 2\}$ for each pair [35]. Pairs are matched by image number rather than sorted position, since the identifiers are not contiguous, and all 2,104 pairs were verified to be one-to-one. Reserving 15% of the official training portion as a validation split, stratified by style with seed 42, gives **899 training**, **159 validation** and **1,046 test** pairs, with styles 303/298/298, 54/52/53 and 619/381/46 respectively.

Two measured properties of FS2K shape the Task 4 experiment design.

First, **the official test set is strongly style-imbalanced**: only 46 of 1,046 test pairs (4.4%) are style 3, against 619 of style 1. Pooled metrics are therefore dominated by style 1, and per-style results are mandatory; any metric requiring a distribution estimate, such as FID, is meaningless on 46 images.

Second, and more subtly, **style is confounded with photograph source in the official training split**. FS2K draws its photographs from three sources, and in the training portion styles 1 and 2 come only from `photo1` (CASIA-WebFace [40], 250×250 tightly cropped faces) while style 3 comes only from `photo2` and `photo3` (studio cut-outs and stock photographs). A generator could therefore learn part of “style 3” from the appearance of the photograph rather than from the condition label. The official test set contains com-

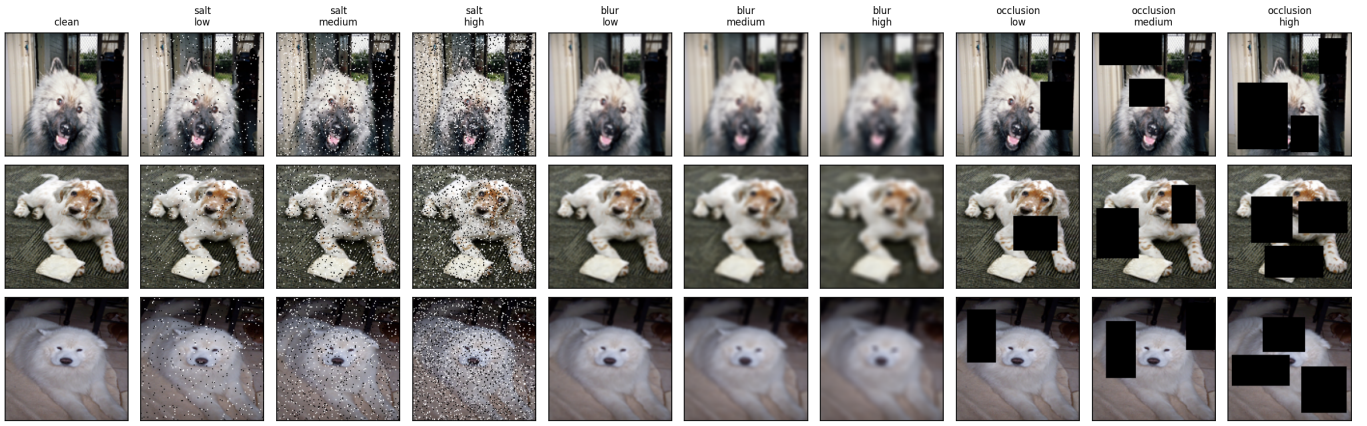


Fig. 1. The four input conditions at the three fixed test severities, applied to three test images by the deterministic test manifest: clean, salt-and-pepper ($p = 0.03, 0.08, 0.15$), Gaussian blur $((3, 0.7), (5, 1.5), (7, 2.5))$ and occlusion (one, two and three rectangles covering 10, 20 and 35%). Every test image appears in all ten variants.

binations never seen in training—179 `photo3` photographs with style-1 sketches—which makes it a direct test of whether the style condition generalises to a new photograph domain, exactly the situation the deployed application faces. Test metrics are therefore reported per style *and* separately for seen and unseen (source, style) combinations. A related artefact: the `photo3` style-3 sketches were drawn on grey paper (90th-percentile grey level 232 against 250–252 elsewhere), so the 179 `photo3` style-1 test pairs carry a background offset of roughly ten grey levels relative to what training taught. The sketches are left unmodified so that metrics are computed against the official ground truth, and the effect is reported rather than normalised away.

All FS2K sketches are exactly greyscale ($R = G = B$ in every pixel of all 2,104 sketches), so they are stored with one channel. Photographs and sketches are made EXIF-upright, composited onto white where transparent, centre-square cropped and resized to 128×128 with antialiased bicubic resampling; values are scaled to $[-1, 1]$ because the generator ends in a $\tanh$. The centre crop is used rather than an anisotropic resize or padding because the non-square sources are exactly the style-3 sources, so either alternative would add a second shortcut correlated with the style label. Spatial augmentation (paired horizontal flip, and the `pix2pix` jitter of upscaling by up to 286/256 and cropping back) is applied to the photograph and sketch as one stacked tensor, so identical geometry is guaranteed by construction.

IV. TASK 1: UNIVERSAL DENOISING AUTOENCODER

A. Methodology

1) *Architecture*: The universal model (`src/models/autoencoder.py`, shared with Tasks 2 and 3) is a convolutional autoencoder. A stem of two 3×3 convolution-BatchNorm-ReLU layers is followed by d stride-2 blocks that halve the resolution and double the channel count, capped at eight times the base width; a 1×1 convolution projects the deepest feature map to c_{lat} channels,

and a mirrored decoder upsamples back to full resolution and ends in a sigmoid.

The depth is fixed at $d = 4$, giving an 8×8 latent. The reason is the occlusion class: the largest test occlusion is a single rectangle of roughly 76×76 pixels, and to synthesise the centre of that hole the network must see past its border on both sides. Measuring the receptive field by back-propagating from a single output pixel gives 175 px at $d = 4$ —the whole image—against 87 px at $d = 3$, which barely spans the hole.

The latent is the **only** path from input to output, asserted by a unit test that checks the output is invariant to changes in the input once the latent is held fixed. Its dimension is $c_{\text{lat}} \times 8 \times 8$ against the $3 \times 128 \times 128 = 49,152$ values of the input, so the compression ratio ranges from $96\times$ at $c_{\text{lat}} = 8$ to $12\times$ at $c_{\text{lat}} = 64$, where the search range deliberately stops: wider latents would keep improving PSNR, since more capacity always helps a pixel metric, but the brief requires a *meaningful* bottleneck.

Upsampling uses bilinear resize followed by convolution rather than transposed convolution. Transposed convolution suffers “uneven overlap” whenever the kernel size is not divisible by the stride, and the unevenness compounds across the two spatial axes into the familiar checkerboard pattern; resize-convolution avoids it because it is implicitly weight-tied in a way that discourages high-frequency artefacts [41]. (An alternative fix initialises sub-pixel convolution to be equivalent to nearest-neighbour resize-convolution [42], [43]; since the decoder here convolves after every upsampling step at every scale, the simpler resize-convolution was used.)

2) *Skip connections*: Unrestricted skips, as in U-Net [11], would let the decoder copy the input around the bottleneck, which the brief rules out; and for this task a full-resolution skip is actively harmful, because here the input *is* the corruption—it hands salt-and-pepper impulses and black occlusion rectangles straight to the decoder. A first model without any skip, however, plateaued at an output quality that was nearly independent of the input and lost about half of the fine detail

even on clean images (the 8×8 latent is the only path for texture). Since skips are standard in restoration and RED-Net attributes real gains to them [12], the final Task 1 models were trained as an ablation that measures their effect rather than assuming it: the selected configuration was trained *three* times with identical hyperparameters and epoch budget—no skip (udae), one skip at 16×16 (udae_skip16) and one at 32×32 (udae_skip32)—and all three were evaluated on the full test manifest. Skips are only ever taken from deep, low-resolution encoder levels, never from full resolution, and the latent remains the only path at the coarsest scale, so every variant keeps a genuine compressed bottleneck. Even the 16×16 skip carries $256 \times 16 \times 16 = 65,536$ values at base width 32 (98,304 at the selected width 48), more than the input itself, which makes the point that a “limited” skip is limited in *resolution*, not in capacity: deep features have passed through many nonlinearities and carry context, whereas a 128×128 skip is two convolutions from raw pixels. The variants are implemented, but the GPU budget did not allow training them (Section IV-B3).

Two diagnostics quantify the trade-off directly. *Impulse survival*, on salt-and-pepper entries, is the fraction of impulse-hit pixels whose output value is still closer to the impulse than to the clean value; it is 1 for the identity and 0 for perfect restoration. *Detail ratio*, on clean entries, is the mean absolute Laplacian of the output luminance divided by that of the target; below 1 means the output is smoother than the clean image, i.e. detail was lost in the bottleneck. Together they separate the two competing effects: skips should raise the detail ratio towards 1 and, if they leak, also raise impulse survival.

3) *Loss*: The loss is the one prescribed by the brief,

$$\mathcal{L}_{\text{UDAE}} = \alpha \mathcal{L}_{L_1}(x, \hat{x}) + (1 - \alpha)(1 - \text{SSIM}(x, \hat{x})), \quad (2)$$

with SSIM computed as specified by Wang et al. [44]: an 11×11 circularly symmetric Gaussian window of standard deviation 1.5 samples, $K_1 = 0.01$, $K_2 = 0.03$, over the valid region and averaged across channels. The L_1 term keeps colours and brightness correct; the SSIM term rewards local structure and contrast. The motivation for mixing them is Zhao et al.’s systematic comparison, which finds that L_2 produces “splotchy” artefacts in flat regions because it tolerates small errors regardless of local structure, that L_1 converges to a better optimum, and that SSIM-type losses are insensitive to uniform biases and therefore permit colour and brightness shifts if used alone [45]. Their recommended mixture uses $\alpha = 0.84$ on the *MS-SSIM* term, chosen so the two contributions are roughly balanced; in the brief’s parameterisation, where α weights L_1 , that corresponds to $\alpha \approx 0.16$. The two conventions must not be confused, and α is tuned rather than assumed, over a range that contains both the brief’s 0.8 and Zhao et al.’s balance point. Single-scale SSIM is used rather than MS-SSIM [46] because at 128×128 a five-level pyramid would reduce the coarsest scale to 8×8 , below the 11×11 window. A perceptual, feature-space loss [47] was also considered and rejected: it optimises a quantity different from the PSNR and SSIM that

TABLE III
OPTUNA SEARCH SPACE OF TASK 1 (STUDY `task1_udae`, TPE SAMPLER, MEDIAN PRUNER, 15 EPOCHS PER TRIAL), THE BRIEF’S STARTING POINT (TRIAL 0) AND THE CONFIGURATION SELECTED BY THE SEARCH (TRIAL 32).

Parameter	Range	Scale	Trial 0	Selected
Learning rate	10^{-4} – 3×10^{-3}	log	10^{-3}	8.28×10^{-4}
Batch size	{16, 32, 64}	cat.	32	16 (edge)
Latent channels	{8, 16, 32, 64}	cat.	32	64 (edge)
Base channels	{16, 32, 48, 64}	cat.	32	48
Dropout	0.0–0.3	uniform	0	0.016
α (L_1 weight)	0.1–0.95	uniform	0.8	0.285
Validation score (3) after 15 epochs			0.5940	0.6263

are reported, and adds a VGG forward pass to every training step on a time-limited GPU.

4) *Training procedure*: Optimisation uses AdamW [48], which applies weight decay to the weights directly rather than through the adaptive gradient, with a fixed decay of 10^{-4} : overfitting is not the principal risk when every epoch sees freshly drawn corruptions. The schedule is one epoch of linear warm-up, which avoids large early Adam steps while the second-moment estimates and the BatchNorm statistics are still noisy [49], followed by per-step cosine decay to 1% of the peak rate [50]. Mixed precision (fp16 autocast with gradient scaling [51]) is used on the GPU, since the T4 has fp16 tensor cores but no bf16 support, with SSIM always evaluated in float32 because its variance terms lose all precision in fp16. Gradients are clipped at norm 1.0 as a safety net, and a non-finite epoch loss aborts the run (or prunes the Optuna trial) rather than silently producing NaNs. Spatial dropout [52], [53] after each convolution block is searched but expected to be of little use, since runtime corruption is already a strong regulariser.

Model selection and the Optuna objective use a validation score independent of the tuned loss weights,

$$\text{score} = 0.5 \text{SSIM} + 0.5 \frac{\text{PSNR}}{40}, \quad (3)$$

so that trials with different α remain comparable—otherwise the search would partly be optimising its own yardstick. Both components are reported separately as well, since PSNR and SSIM measure different things and can disagree [54].

5) *Optuna search design*: The study `task1_udae` maximises (3) on the validation manifest with a TPE sampler [55], [56] seeded at 42 after ten random start-up trials, and a median pruner [57] that is inactive for the first five trials and the first two epochs of each trial. Trial 0 is the brief’s own starting configuration, so the report can state how much the search gained over it. The search space is given in Table III.

Trials run 15 epochs, a quarter of the 60-epoch final run, and the cosine schedule anneals completely within each trial so that configurations are compared as fully annealed models rather than mid-schedule. The known limitation of this low-fidelity protocol is that settings which pay off only late—a lower learning rate, or dropout—can be under-rated. The

ranges, in the order of Table III: the learning rate brackets Adam’s usual 10^{-3} ; the batch sizes give 184, 92 or 46 optimiser steps per epoch on 2,944 images; the latent widths give $96\times$ to $12\times$ compression; the base widths give 1.0 to 16.5M parameters; the dropout range stays low because runtime corruption already regularises; and the α range spans SSIM- to L_1 -dominated losses.

6) *Search results*: The study was launched for 40 trials with a 90-minute cap, and the cap ended it: 33 trials were run (15 completed, 18 stopped by the median pruner) in 89.6 minutes of trial time on the T4. The best trial was the *last* one, trial 32, with a validation score of 0.6263 against 0.5940 for the brief’s starting configuration (Fig. 2, left), so the search was still improving when it stopped and more trials could plausibly have helped. Four observations explain the selected configuration.

Loss weighting. Optuna moved α from the brief’s 0.8 to 0.285, putting most of the weight on the $(1 - \text{SSIM})$ term, and α is the most important parameter in Optuna’s importance estimate over the completed trials (0.44, Fig. 2, right). This agrees with Zhao et al. once their parameterisation is read correctly: their $\alpha = 0.84$ weights the MS-SSIM term [45], which corresponds to $\alpha \approx 0.16$ here, not to the brief’s 0.8 on L_1 . One caveat applies: the selection score (3) is itself half SSIM, which favours SSIM-heavy losses.

Bottleneck at the edge of the range. The best trial used the widest latent in the range, $64 \times 8 \times 8 = 4,096$ values ($12\times$ compression). A wider bottleneck passes more detail, so reconstruction quality rises with it; the margin is small here (the best 32-channel trial, trial 24, scored 0.6225) and the latent width has a low importance (0.02), but the direction is the expected one. The range was capped deliberately: much wider latents approach an identity mapping, which the brief rules out by requiring a meaningful bottleneck, so the edge result reflects a quality-versus-compression trade-off rather than an oversight of the search.

Short-budget bias. The smallest batch size won, and TPE converged on it (11 of the last 12 trials used batch 16). With a fixed 15-epoch budget, batch 16 gives four times as many optimiser steps as batch 64 (184 against 46 per epoch), so short trials favour configurations that learn fast; the final run, four times longer, mitigates but does not remove this bias.

Dropout. Dropout has the second-highest importance (0.40), but in the negative direction: the five best trials all used a rate of at most 0.065 and the selected value is 0.016, consistent with the expectation that runtime corruption already regularises the model.

7) *Baselines*: Two reference points accompany every Task 1 result. The *identity* baseline is the corrupted input itself and measures how damaging each corruption is. The *oracle classical* baseline applies, to each entry, the textbook method for its corruption *using information the network never receives*: a 3×3 median filter for salt-and-pepper [58], [59]; unsharp masking with the true kernel (k, σ) , $\hat{x} = y + (y - h * y)$, which is one step of Van Cittert-style iterative deconvolution [60], [61]; and Telea fast-marching inpainting of the true rectangle

mask [62], [63], [64]. Clean inputs are returned unchanged, so the oracle is exact there. Because the oracle is given the corruption type and its exact parameters, beating it is a strong result and losing to it on one corruption is an informative finding rather than a failure to conceal.

B. Results

1) *Final training*: [TODO: final training of the three variants on Kaggle: parameters, epochs, best epoch and validation score per variant, epoch time; interpret the training curves]

2) *Test results*: [TODO: test results of the deployed variant by type and severity with the identity and oracle-classical baselines; interpret]

3) *Skip-connection ablation*: [TODO: interpret Table VI: PSNR/SSIM, impulse survival and detail ratio of udae, udae_skip16, udae_skip32; which variant is deployed and why (validation score), and whether any skip leaks the corruption]

C. Failure analysis

[TODO: failure analysis from the new failure-case figure]

V. TASK 2: CORRUPTION CLASSIFICATION AND HARD ROUTING

A. Methodology

1) *Classifier*: The corruption classifier (`src/models/classifier.py`) maps an RGB image to four logits. It is a conventional convolutional classifier [65], [66] with batch normalisation [67]: each stage applies one or two 3×3 convolution-BatchNorm-ReLU layers followed by 2×2 max pooling, and two design choices follow directly from what the cues look like. The first stage runs at full 128×128 resolution, because the evidence is high-frequency: a few isolated impulse pixels, or the *absence* of high frequencies under mild blur, would both be destroyed by early downsampling. The head concatenates global average and global max pooling, because the cues are sparse—a handful of bright pixels, or one black rectangle—and average pooling alone dilutes them.

Training uses plain multiclass cross-entropy, as the brief requires, with no label smoothing: the probabilities are displayed in the application and are used to initialise the Task 3 gate, so they should be the model’s own estimates. Batches are exactly class-balanced by the sampler of Section III, and the labels come free from the corruption pipeline. AdamW is used with weight decay applied only to convolution and linear weights, not to BatchNorm parameters or biases, where decaying them does not regularise what the normalised layer computes [68].

Model selection, early stopping and the Optuna objective all use validation **macro-F1** rather than loss or accuracy. Routing uses the `argmax`, so the decision metric is what matters; macro-averaging weights the four classes equally and penalises a class that is over-predicted, which accuracy on a balanced set would hide [69]. Validation cross-entropy is logged but not used for selection, because it can rise through growing over-confidence while accuracy still improves [70].

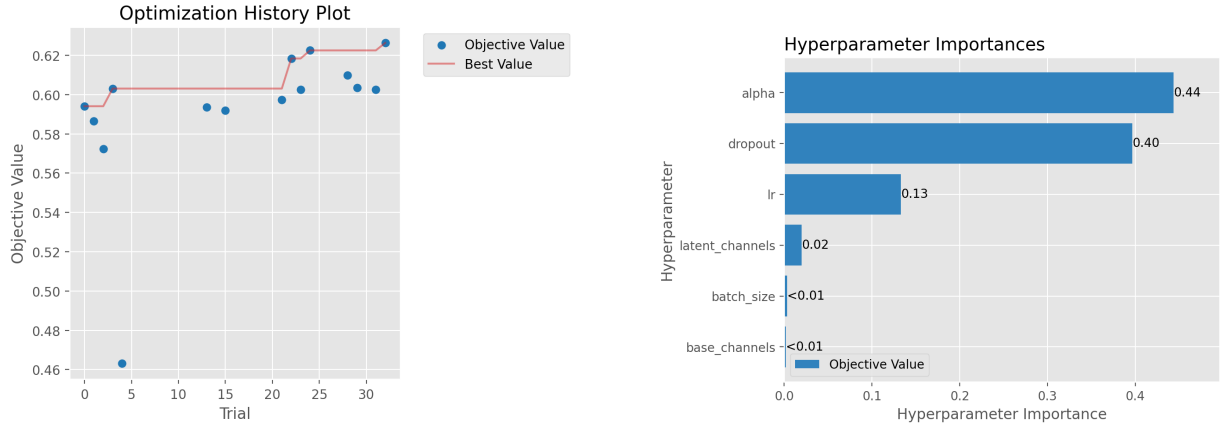


Fig. 2. Optuna study `task1_udae`. Left: validation score of every completed trial and the running best; pruned trials have no final value and are not shown. Right: parameter importances estimated from the completed trials.

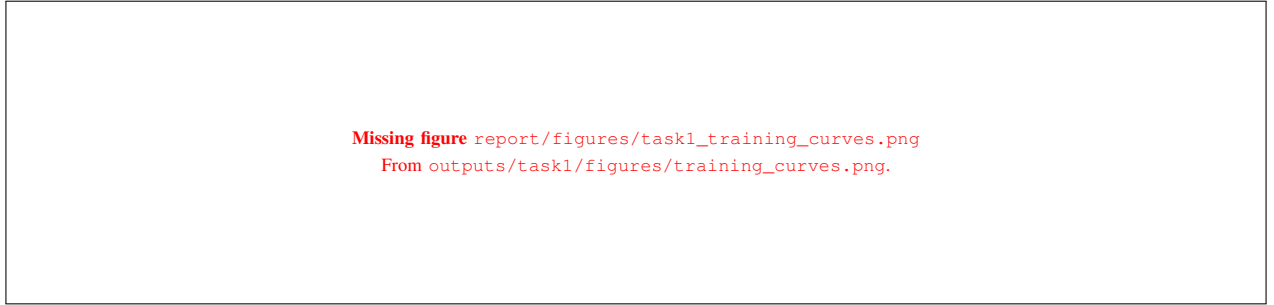


Fig. 3. Task 1 training: training and validation loss (left), and validation PSNR (centre) and SSIM (right) per input condition on the validation manifest. The dotted line marks the best epoch (59).

TABLE IV

TASK 1 TEST RESULTS PER INPUT CONDITION (36,690 ENTRIES; 3,669 CLEAN AND 11,007 PER CORRUPTION): PSNR (DB) AND SSIM OF THE CORRUPTED INPUT ITSELF, OF THE ORACLE CLASSICAL BASELINES THAT ARE TOLD THE CORRUPTION AND ITS PARAMETERS, AND OF THE UDAE, WHICH IS TOLD NOTHING. BOTH BASELINES RETURN CLEAN INPUTS UNCHANGED (EXACT, ∞); THE POOLED “CORRUPTED” ROW COVERS THE THREE CORRUPTIONS ONLY.

Type	Corrupted input	Oracle classical	UDAE
clean	—	—	—

Missing table report/tables/task1_comparison_by_type.tex, copied from outputs/task1/tables/comparison_by_type.tex

2) *Specialists*: Three specialist autoencoders are trained, one per corruption, each seeing *only* its own corruption with a fresh severity on every load and the clean image as target. The three runs are fully independent—different seeds, separate optimisers, separate checkpoints—so their parameters are independently trained as the brief requires. They share the architecture of Section IV. A first set of specialists with a pure bottleneck produced over-smoothed outputs and blurry occlusion fills, so the final specialists add *one* limited skip at 32×32 (configs/task2_specialists_skip.yaml), one level above their 16×16 latent—the counterpart of Task 1’s 16×16 skip on its 8×8 latent. No full-resolution skip is used, so a specialist cannot pass its corruption’s pixel-level signature straight to the output; the latent remains the only path at the coarsest scale.

To keep the experiment computationally feasible the brief

permits one shared architecture search. Each trial trains all three specialists with the trial’s hyperparameters, *interleaved epoch by epoch*, and reports the running mean of the three validation scores to the pruner. Interleaving matters: training them sequentially would show the pruner only the salt-and-pepper score for the first third of each trial, so a configuration that suits impulse noise but fails on occlusion would survive longer than it deserves. After the study the three specialists are retrained independently on the full schedule.

Unlike Task 1, the specialist search may also choose the latent *resolution*: a 16×16 latent (encoder depth 3) keeps more spatial layout than an 8×8 latent with the same number of values. The six presets, from $8 \times 8 \times 32$ to $16 \times 16 \times 64$, hold 2,048 to 16,384 values ($24\times$ to $3\times$ compression), so every option is still a genuine bottleneck.

TABLE V

TASK 1 TEST RESULTS PER INPUT CONDITION AND SEVERITY LEVEL (3,669 ENTRIES PER ROW), WITH THE SAME TWO REFERENCE POINTS AS TABLE IV.

Type	Severity	Corrupted input		Oracle classical		UDAE	
salt	low	—	—	—	—	—	—

Missing table report/tables/task1_comparison_by_type_level.tex, copied from outputs/task1/tables/comparison_by_type_level.tex

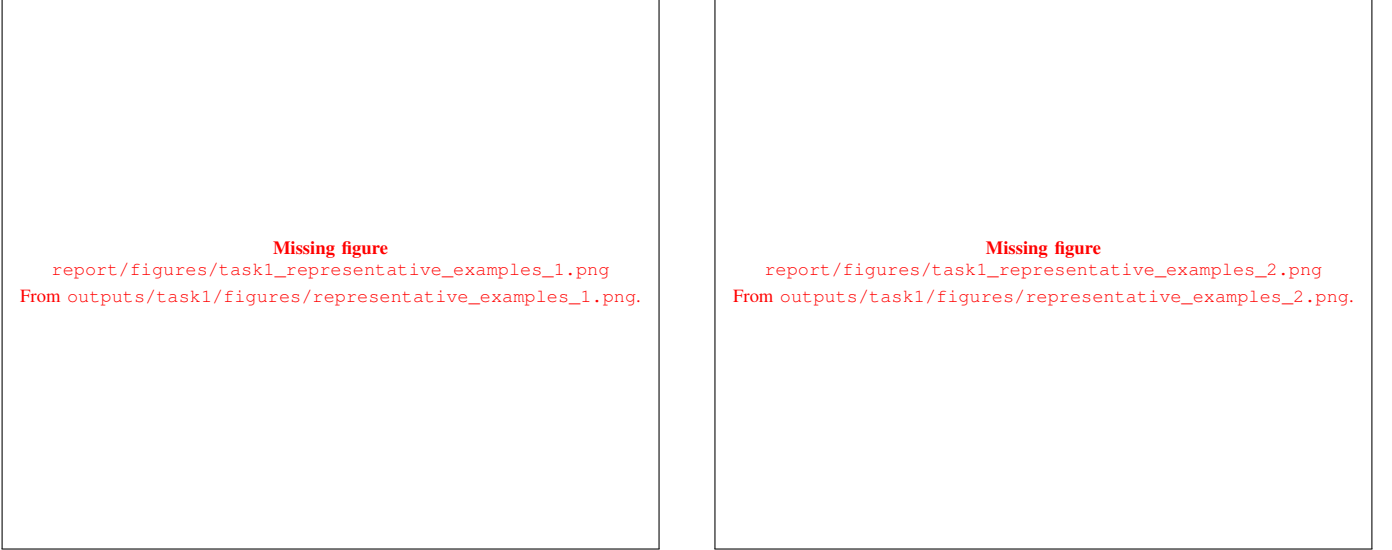


Fig. 4. Representative Task 1 test restorations, rows of target, corrupted input, UDAE output and absolute error map, labelled with output PSNR (input PSNR) and SSIM. Left: three clean entries and salt-and-pepper low/medium/high; right: blur and occlusion at low/medium/high. Each entry is the one closest to its group’s median SSIM, so the examples are typical rather than chosen by hand.



Fig. 5. Task 1 restoration quality against severity: PSNR (top) and SSIM (bottom) of the UDAE, the oracle classical baselines and the corrupted input for each corruption; the dotted line is the UDAE’s score on clean inputs. The UDAE curve is almost flat at the clean-input level for salt-and-pepper and mild blur—the ≈ 25 dB ceiling.

TABLE VI

SKIP-CONNECTION ABLATION: ONE LIMITED SKIP ADDED AT THE STATED RESOLUTION, SAME HYPERPARAMETERS AND EPOCH BUDGET AS THE MAIN MODEL.

3) *Hard routing*: At inference the classifier produces $p = \text{softmax}(C(\tilde{x}))$ and the route is $r = \arg \max_k p_k$. The restorer starts from a copy of the input and overwrites only the entries with $r > 0$ using the corresponding specialist, which has three consequences worth stating: a clean-routed input is returned *bit-identical* to the input through the identity bypass, never unnecessarily processed by a restoration expert; each expert runs at most once per batch on its own sub-batch rather than

once per sample; and all models run in evaluation mode, so BatchNorm uses running statistics and a sample’s output does not depend on the rest of its batch.

The system is evaluated in both modes the brief prescribes. *Oracle routing* takes r from the manifest label and isolates the restoration ability of the specialists; *predicted routing* uses the classifier and measures the complete operational system. Correctly routed entries give identical metrics in both modes, which is asserted by a test.

4) *Optuna search design*: Two studies are run (Table VII): `task2_classifier` (30 trials of 12 epochs, objective validation macro-F1, 45-minute cap) and `task2_specialists` (20 trials of 6 epochs per specialist, objective the mean of the three best validation restoration

Missing figure report/figures/task1_failure_cases.png
From outputs/task1/figures/failure_cases.png.

Fig. 6. Task 1 failure cases, selected automatically: the lowest-SSIM output for each input condition (rows 1–4) and the two corrupted entries on which the UDAE loses most PSNR relative to its own input (rows 5–6). Labels give output PSNR (input PSNR) and SSIM.

TABLE VII
OPTUNA SEARCH SPACES FOR TASK 2 AND THE SELECTED CONFIGURATIONS (BEST TRIALS 1 AND 10), COMMITTED AS CONFIGS/TASK2_CLASSIFIER.YAML AND CONFIGS/TASK2_SPECIALISTS.YAML.

Parameter	Range	Selected
<i>Classifier</i> (task2_classifier, 30 trials $\times$ 12 epochs)		
Learning rate	10^{-4} – 3×10^{-3} (log)	1.70×10^{-3}
Batch size	{32, 64, 128}	32
Channels	four presets	32–64–128–256
Convs per stage	{1, 2}	1
Dropout (head)	0.0–0.5	0.146
Weight decay	10^{-5} – 10^{-1} (log)	2.9×10^{-4}
Validation macro-F1		0.9959
<i>Specialists</i> (task2_specialists, 20 trials $\times$ 6 epochs)		
Learning rate	10^{-4} – 3×10^{-3} (log)	2.37×10^{-3}
Batch size	{16, 32, 64}	16 (edge)
Base channels	{16, 32, 48}	48
Latent shape	six presets	$16 \times 16 \times 32$ (6 $\times$)
α (L_1 weight)	0.4–0.95	0.516
Mean validation score		0.6819

scores, 80-minute cap). Both use the TPE sampler seeded at 42 with a median pruner. The classifier’s channel presets are 16–32–64–128, 32–64–128–256, 48–96–192–384 and the five-stage 32–64–128–256–256, which trade width against depth at 0.21–1.89 GMAC per image; the batch sizes are multiples of four so that every batch is exactly balanced.

5) *Search results: Classifier.* All 30 trials ran, and took only 15.6 minutes of trial time, well inside the cap: the median pruner stopped 25 of them, mostly after their fourth epoch, and only five completed. The best trial was trial 1, with validation macro-F1 0.9959 (lr 1.70×10^{-3} , batch 32, channels 32–64–128–256, one convolution per stage, dropout 0.146, weight decay 2.9×10^{-4}). The pruning rate is itself a finding:

the objective is nearly saturated. Even pruned trials had reached macro-F1 of about 0.98 within three or four epochs (trial 29: 0.9783), and the five completed trials span only 0.992–0.996 (Fig. 7, left). Detecting the corruption type is easy for a small CNN on this data, so the search mostly selected an *inexpensive* configuration—the narrower four-stage network with one convolution per stage, 0.39 M parameters, which is also the cheaper gate for Task 3. With five completed trials so close together, the importance estimate of this study is not informative and is not used.

Specialists. Exactly 20 trials finished, 12 completed and 8 pruned, none failed, in 61.5 minutes of trial time. The best was trial 10, with a mean validation score of 0.6819 (salt-and-pepper 0.7006, blur 0.7295, occlusion 0.6155), and its configuration is shared by the three specialists: lr 2.37×10^{-3} , batch 16, base width 48, latent $16 \times 16 \times 32 = 8,192$ values (6 $\times$ compression) and $\alpha = 0.516$; each specialist has 3.97 M parameters. The learning rate dominates the importance estimate (0.76), followed by α (0.20; Fig. 7, right). The study also gives a free estimate of run-to-run noise. The first session was stopped after about eleven minutes, while the checkpoint incident described in the AI-use appendix was being fixed; trials 0–2 had been saved and the resumed study continued from trial 3. Because the TPE sampler was re-created with the same seed, it proposed the parameters of trials 0–2 again as trials 3–5, and the repeated runs differ by only $2\text{--}4 \times 10^{-4}$ in score (0.6046 against 0.6042, 0.6045 against 0.6047). Score differences of that size between trials are therefore noise: trials 10, 12 and 13 (0.6819, 0.6810, 0.6806), which share every categorical choice and differ only slightly in learning rate and α , are effectively tied, so the selected configuration is a robust region rather than a single lucky trial. Three of the 20 trials were spent on these replays, a cost of the resume logic worth fixing.

Three observations carry over from Task 1. *Loss weighting:* α again settled well below the brief’s 0.8 (0.516; Task 1 chose 0.285), so the searches consistently moved weight towards the SSIM term. *Short-budget bias:* the smallest batch size won again. *Speed of learning:* after only six epochs, the best trial’s blur specialist already slightly exceeded the fully trained universal model on the same 184 validation blur entries (score 0.7295 against 0.7245 after 60 epochs).

A confound for the cross-task comparison. The specialists selected a latent twice as large as Task 1’s: 8,192 values (6 $\times$ compression) against 4,096 (12 $\times$). Both are genuine bottlenecks, but part of any advantage of hard routing over the universal model may come from the larger bottleneck rather than from specialisation alone; the Task 1 search range, capped at 4,096 values to keep a meaningful bottleneck, did not offer the larger latent. The comparisons in Sections V and VI are read with this in mind. The three specialists together have 11.9 M parameters, against 9.31 M for the UDAE.

6) *Final training:* The classifier was retrained for 60 epochs (early-stopping patience 15) and each specialist for 40 epochs (patience 10), independently and with its own seed, from the committed configurations. These runs were first made on

Missing figure
report/figures/task2_classifier_
optuna_intermediate_values.png
From outputs/task2/optuna/task2_
classifier/intermediate_values.png.

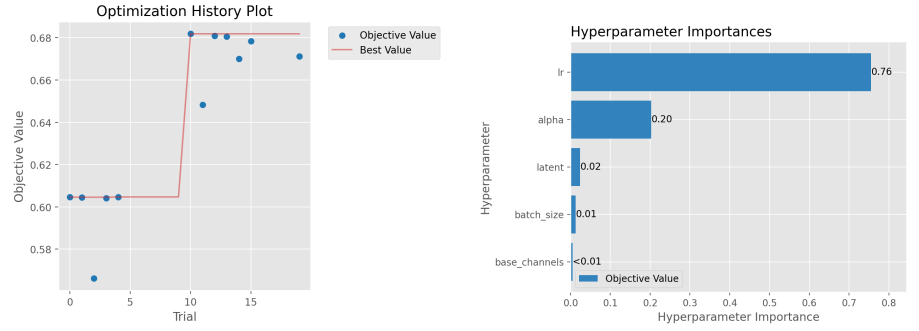


Fig. 7. Optuna studies of Task 2. Left: validation macro-F1 per epoch of every classifier trial; all trials approach 1.0 within a few epochs, and the median pruner stops most of them at the fourth epoch. Centre and right: the specialist study’s optimisation history and parameter importances.

TABLE VIII
TASK 2 CLASSIFIER ON THE TEST MANIFEST: PER-CLASS PRECISION, RECALL AND F1 (3,669 CLEAN ENTRIES AND 11,007 PER CORRUPTION), MACRO AVERAGES AND OVERALL ACCURACY.

Class	Precision	Recall	F1	n
clean	—	—	—	3669
salt	—	—	—	11007
blur	—	—	—	11007
occlusion	—	—	—	11007
macro avg	—	—	—	36690
accuracy	—	—	—	36690

Missing table report/tables/task2_classifier_test_per_class.tex, copied from outputs/task2/tables/classifier_test_per_class.tex

Missing figure report/figures/task2_classifier_
confusion_test.png
From outputs/task2/figures/classifier_confusion_
test.png (Kaggle run): row-normalised 4×4 confusion matrix,
rows = true class (clean, salt, blur, occlusion), counts in brackets.

Fig. 8. Normalised confusion matrix of the corruption classifier on the test manifest. Rows are true classes and sum to one, so each row is a recall profile; this is the same normalisation used for the Task 3 routing matrix, which makes the two directly comparable.

Colab and are being repeated on Kaggle, with identical configurations, after the Colab GPU quota ran out (Section VIII); the results below come from the Kaggle run. [TODO: best epoch and validation macro-F1 of the final classifier, best epoch and validation score of each specialist, and epoch times on the Kaggle T4, from the Kaggle logs or Weights & Biases]

B. Results

[TODO: paragraph interpreting Table VIII and Figure 8 from the Kaggle run: overall accuracy and macro-F1, which

Missing figure
report/figures/task2_classifier_blur_detection.png
From outputs/task2/figures/classifier_blur_detection.png
(Kaggle run): P(blur) and P(clean) against the applied blur strength $s(k, \sigma)$ with 95% Wilson intervals; strength 0 = clean photographs.

Fig. 9. Clean-versus-blur discrimination as a function of the true blur strength. The analysis uses the applied-kernel strength rather than σ , for the reason given in Section III.

confusions dominate, and whether they are concentrated at the low severity levels (outputs/task2/tables/classifier_test_by_type_level.csv)]

[TODO: paragraph on the two expected confusion mechanisms, with numbers: (i) naturally soft or low-resolution clean photographs being called blurred, quantified against the variance of the Laplacian as a focus measure [71] with Wilson intervals [72]; (ii) dark image content being called occlusion, and black occlusion rectangles on already-dark fur going undetected]

[TODO: paragraph interpreting Table IX: the specialists’ own quality under oracle routing against the UDAE of Table V (bearing in mind the bottleneck confound above), and how close predicted routing comes to oracle routing]

Clean inputs and the PSNR cap. Under oracle routing every clean input passes through the identity bypass, so the output equals the target exactly: zero MSE, SSIM of one, and a PSNR that is infinite and stored at the implementation’s 100 dB cap. These entries are reported as “exact (n/n)” rather than averaged as 100 dB, and the pooled restoration figures use the *corrupted* rows only. Under predicted routing the share of clean inputs that remain exact equals the classifier’s clean recall, and misrouted clean photographs receive a finite PSNR

TABLE IX

HARD-ROUTED RESTORATION ON THE TEST MANIFEST: ORACLE ROUTING (TRUE LABEL) AGAINST PREDICTED ROUTING (CLASSIFIER) PER CORRUPTION TYPE AND SEVERITY, WITH ROUTING ACCURACY, PSNR AND SSIM IN BOTH MODES, THE NUMBER OF MISROUTED ENTRIES AND THE NUMBER OF HARMFUL MISROUTINGS (PREDICTED ROUTING AT LEAST 1 DB WORSE THAN ORACLE ROUTING).

Type	Level	n	Route acc.	PSNR or.	PSNR pr.	SSIM or.	SSIM pr.	Misr.	Harmful
clean	none	3669	—	exact	—	1.0000	—	—	—
salt	low	3669	—	—	—	—	—	—	—

Missing table report/tables/task2_routing_oracle_vs_predicted.tex, copied from outputs/task2/tables/routing_oracle_vs_predicted.tex

Missing figure report/figures/task2_routing_failures.png
From outputs/task2/figures/routing_failures.png (Kaggle run):
worst harmful misroutings per confusion pair, showing target, input, oracle output,
predicted output and the four classifier probabilities.

Fig. 10. Cases where a classifier error causes a restoration failure.

TABLE X

EVERY CLASSIFIER CONFUSION ON THE TEST MANIFEST: NUMBER OF ENTRIES, SHARE OF THE TRUE TYPE, MEAN PSNR AND SSIM CHANGE OF PREDICTED RELATIVE TO ORACLE ROUTING, AND HOW MANY MISROUTINGS WERE HARMFUL, NEUTRAL OR BENEFICIAL (± 1 DB).

True	Routed to	n	Δ PSNR	Harmful / neutral / benef.
clean	blur	—	—	—

Missing table report/tables/task2_routing_misrouting.tex, copied from outputs/task2/tables/routing_misrouting.tex

that is reported separately.

C. Failure analysis

[TODO: discussion of Table X and Figure 10: for each (true, routed-to) pair, how many entries, at which severities, and the mean PSNR change relative to oracle routing. Separate harmful misroutings (≥ 1 dB worse) from neutral and *beneficial* ones: a mild blur routed to the identity can score better than the blur specialist’s reconstruction if the bottleneck loses more detail than the blur itself removed. Report beneficial misroutings explicitly rather than omitting them, and relate the clean-to-occlusion confusions to dark image content (outputs/task2/tables/classifier_dark_regions.csv) and the clean-to-blur confusions to natural softness (outputs/task2/figures/classifier_sharpness.png)]

VI. TASK 3: SOFT MIXTURE-OF-EXPERTS RESTORATION

A. Methodology

1) *Model*: The soft mixture replaces the argmax of Task 2 with a temperature-scaled softmax over the same four

branches:

$$w = \text{softmax}\left(\frac{G(\tilde{x})}{\tau}\right), \quad (4)$$

$$\hat{x} = w_0 \tilde{x} + w_1 A_{\text{salt}}(\tilde{x}) + w_2 A_{\text{blur}}(\tilde{x}) + w_3 A_{\text{occ}}(\tilde{x}). \quad (5)$$

The gate G is the Task 2 classifier and the three experts are the Task 2 specialists, loaded from their checkpoints and rebuilt from their own stored configurations, so whatever architecture the Task 2 search selected is used unchanged. Branch 0 is a *true identity*, not a learned clean expert: a clean input needs only the gate to place its weight on branch 0 to be reproduced perfectly, so there is no reconstruction floor for clean images.

The temperature τ is a registered buffer rather than a parameter or a Python float, which means it is saved with the state dictionary, is never touched by the optimiser, and is traced into the exported ONNX graph as a constant, so the deployed model provably uses the temperature it was trained with. Temperature scaling of a softmax follows Hinton et al. [73]: a higher T produces a softer distribution. Here a small τ approaches the hard routing of Task 2 and a large τ approaches uniform blending, so τ is the single knob that interpolates between Tasks 2 and 3—which is why it is searched. (A stochastic hard route with a differentiable relaxation, as in the Gumbel-softmax [74], was the alternative not taken: the brief asks for a continuous weight on every branch, and a dense blend needs no relaxation.) The gate logits and the softmax are computed in float32 even under mixed precision, since the routing decision multiplies *every* branch output.

A caveat that should be stated plainly. Equation (5) trained on the error of the blend is exactly the *cooperative* error function of Jacobs et al. [14], the formulation they argue “does not encourage localization” because each expert need only cancel the residual left by the others. Two features of this system counteract that. The experts are pre-trained to specialise rather than learned from scratch, and the cross-entropy term below supervises the routing directly with the known runtime corruption label. Without both, the blend objective would be expected to drift towards using many experts for every input.

2) *Loss*:

$$\mathcal{L}_{\text{MoE}} = \lambda_1 \mathcal{L}_{L_1} + \lambda_s (1 - \text{SSIM}) + \lambda_c \mathcal{L}_{\text{CE}} + \lambda_b \mathcal{L}_{\text{balance}}. \quad (6)$$

The cross-entropy is applied to the *raw* gate logits $G(\tilde{x})$ rather than to $G(\tilde{x})/\tau$. With the scaled logits, the same parameter τ would control both the sharpness of the routing and the magnitude of the classification gradient—a small τ multiplies

the cross-entropy gradient by $1/\tau$ —so the searches over τ and λ_c would be confounded. Keeping the cross-entropy on the raw logits makes τ purely a routing knob, λ_c purely a weight, and preserves the Task 2 objective so that the gate remains a calibrated corruption classifier whose weights are interpretable in the gating analysis and in the application display.

3) *The balance regulariser*: The regulariser is the brief’s,

$$\mathcal{L}_{\text{balance}} = \sum_{k=0}^3 \left(\bar{w}_k - \frac{1}{4} \right)^2, \quad (7)$$

with $\bar{w}_k$ the mean weight of branch k over the batch. Three arguments support it, and the alternatives were examined in detail before it was kept.

First, it is the appropriate simplification for a dense gate. Shazeer et al. use two terms [17]: an importance loss $\mathcal{L}_{\text{imp}} = w_{\text{imp}} \cdot \text{CV}(\text{Importance}(X))^2$, the squared coefficient of variation of the batchwise sum of gate values, and a load loss $\mathcal{L}_{\text{load}} = w_{\text{load}} \cdot \text{CV}(\text{Load}(X))^2$ over a smooth estimator of the number of examples routed to each expert. The Switch Transformer merges them into

$$\mathcal{L}_{\text{switch}} = \alpha N \sum_{i=1}^N f_i P_i, \quad (8)$$

where f_i is the fraction of tokens dispatched to expert i and P_i the mean router probability, with $\alpha = 10^{-2}$ chosen after a sweep over five orders of magnitude [18], [19]. Both the load term and f_i exist to count *discrete dispatches* in a top- k router. The gate here is dense: all four branches always run, nothing is dropped, and there is no load to count, only importance. With soft routing f and P coincide, and since $\sum_k (P_k - 1/N)^2 = \sum_k P_k^2 - 1/N$, equation (7) is the Switch loss up to an affine transformation. The brief’s term is therefore not a weaker substitute for the published losses; for a dense gate it is the same objective.

Second, it does not fight the supervised objective. Because training batches contain exactly $B/4$ images of each condition, perfect routing—one-hot on the true class—gives every branch a mean weight of exactly $1/4$, so $\mathcal{L}_{\text{balance}} = 0$ at the optimum of the cross-entropy. The two terms are compatible by construction, a property that follows from the balanced sampler and is unavailable in the MoE literature, which has no labels. (With an unbalanced batch the term would actively punish correct routing, which a unit test asserts.) Total collapse onto a single branch costs $(1 - 1/4)^2 + 3 \cdot (1/4)^2 = 0.75$.

Third, the exact form matters less than its presence. Shazeer et al.’s own ablation, reproduced in Table XI, shows that any combination containing at least one balance term gives nearly identical model quality over two orders of magnitude of weight, whereas omitting them entirely is far worse.

Entropy regularisers were considered and rejected as a *loss*. Maximising routing entropy would push against the cross-entropy by rewarding weight spread onto wrong branches; minimising it would push towards hard routing, which is Task 2. (The information-maximisation combination of a high marginal entropy and a low conditional entropy [75], [76]

TABLE XI
EFFECT OF THE BALANCE LOSSES IN SHAZEER ET AL. [17], TABLE 6 (MoE-256 MODEL, 10 EPOCHS). ANY BALANCE TERM SUFFICES; NONE IS MUCH WORSE. CV DENOTES THE COEFFICIENT OF VARIATION.

w_{imp}	w_{load}	Perplexity	CV(Imp.)	CV(Load)	$\frac{\max}{\text{mean}}$
0.0	0.0	39.8	3.04	3.01	17.80
0.2	0.0	35.6	0.06	0.17	1.47
0.0	0.2	35.7	0.22	0.04	1.15
0.1	0.1	35.6	0.06	0.05	1.14
0.01	0.01	35.7	0.48	0.11	1.37
1.0	1.0	35.7	0.03	0.02	1.07

remains the principled upgrade if a stronger regulariser is ever needed, and confidence penalties [77] the remedy if the gate saturates.) Entropy is used here as a *diagnostic* instead, reported per class. The one blind spot of (7) is that it sees only batch-mean weights and is therefore insensitive to a gate that routes each class confidently to the *wrong* branch; the cross-entropy term and the routing matrix below exist for that.

4) *Routing collapse*: Collapse is defined once and used identically by the Optuna pruner, the per-epoch log and the final evaluation. From the routing matrix on the balanced validation manifest, $R[c, k]$ being the mean weight of branch k over entries of true class c (rows sum to one), define $\text{usage}_k = \text{mean}_c R[c, k]$, which is $1/4$ under balanced perfect routing, and $\text{off}_k = \text{mean}_{c \neq k} R[c, k]$. A configuration has collapsed if some branch is *dead* ($\text{usage}_k < 0.05$, a fifth of its fair share; since $\text{usage}_k \geq R[k, k]/4$ such a branch receives less than 0.2 of the weight even on the inputs it exists for) or *dominating* ($\text{off}_k > 0.5$, taking more than half the weight on average on the other three classes). The criterion deliberately does not flag a merely uncertain model: uniform routing is informative, not useless, and the tests assert that uniform weights are *not* flagged while total collapse is flagged as both dead and dominating.

5) *Staged training*: Training proceeds in the two stages the brief prescribes. In the **warm-up** the experts are frozen in two senses at once: their parameters have `requires_grad = False` and the warm-up optimiser is built over the gate only, and they are kept in evaluation mode with the `train()` method overridden so that a later call cannot silently undo it. The second part is not redundant—without it, BatchNorm would keep updating its running statistics on every forward pass and dropout would stay active, so the “frozen” experts would drift and the warm-up would no longer be a clean measurement of the gate. In the **joint stage** all parameters train, with the gate at learning rate η and the experts at 0.1η , under cosine decay.

The schedule’s shape has direct empirical support. Kumar et al. [78] show that full fine-tuning from a randomly initialised head distorts good pretrained features, because the lower layers move while the head is still wrong, and that linear probing followed by full fine-tuning combines the benefits of both. The warm-up is exactly the probe stage, with the gate as the head: it prevents a miscalibrated gate from pushing destructive gradients into three specialists that are already good. The lower

TABLE XII
OPTUNA SEARCH SPACE FOR TASK 3 (STUDY `task3_moe`); TRIAL 0 IS THE BRIEF’S STARTING CONFIGURATION.

Parameter	Range	Rationale	Selected
Joint learning rate	10^{-5} – 3×10^{-4}	log; capped at the warm-up rate	–
Temperature τ	0.3–3.0	log; near-hard to near-uniform	–
λ_c	0.01–1.0	log; a decade either side of 0.1	–
λ_b	10^{-3} – 10^{-1}	log; contains Switch’s 10^{-2}	–
α ($\lambda_1 = \alpha$, $\lambda_s = 1 - \alpha$)	0.5–0.95	constant reconstruction scale	–

[TODO: fill the Selected column from `outputs/task3/best_config.yaml` and state the best trial, its validation score and the number of completed, pruned and collapse-pruned trials (`outputs/task3/optuna/task3_moe/trials.csv`)]

expert learning rate in the joint stage is discriminative fine-tuning [79], and is additionally motivated by the fact that the gradient reaching an expert is scaled by its routing weight, so a rarely selected expert receives little signal while a dominant one receives a lot. The experts’ BatchNorm running statistics are kept fixed during joint training, since an expert’s input distribution changes once it sees all four conditions and re-estimating would move its normalisation away from the regime it was trained in [80].

The validation objective caps per-image PSNR at 40 dB *inside the objective only*, because a clean input routed to the identity reaches 60–100 dB and would otherwise dominate model selection over actual restoration quality. The uncapped score is logged, and reported test metrics are never capped this way.

6) *Optuna search design*: The study `task3_moe` searches the five parameters the brief names (Table XII), with trial 0 enqueued at the brief’s starting values $\lambda_1 = 0.8$, $\lambda_s = 0.2$, $\lambda_c = 0.1$, $\lambda_b = 0.01$. Pruning has two paths, both recorded in the trial attributes: the median pruner for weak configurations, and the collapse criterion above—a task-specific stopping rule that no generic pruner can express, and which the brief explicitly permits. A diverged loss is converted into a prune with its reason rather than a crashed trial, so one bad learning rate cannot end a session.

Budget actually used. The study was planned as 20 trials of one warm-up and four joint epochs, followed by a final run of 2 warm-up and 20 joint epochs. Because the Colab GPU quota ran out on the deadline day, Task 3 was trained on Kaggle with a reduced budget: **6 trials of one warm-up and three joint epochs** (25-minute cap), and a **final run of 2 warm-up and 10 joint epochs**. Six trials are the brief’s reference configuration plus five random proposals—the TPE sampler only starts modelling after ten random start-up trials—so this search is a small random search around the reference point rather than a model-based optimisation, a limitation stated in Section X.

TABLE XIII
SOFT MIXTURE-OF-EXPERTS RESTORATION ON THE TEST MANIFEST, PER INPUT TYPE AND SEVERITY. PSNR* EXCLUDES OUTPUTS IDENTICAL TO THE TARGET (PSNR CAPPED AT 100 dB); “EXACT” COUNTS THEM.

Type	Level	n	PSNR	PSNR*	SSIM	Exact
clean	none	3669	–	–	–	–
salt	low	3669	–	–	–	–

Missing table `report/tables/task3_test_by_type_level.tex`, copied from `outputs/task3/tables/test_by_type_level.tex`

B. Results

[TODO: final training on Kaggle (2 warm-up + 10 joint epochs): best epoch, validation score, epoch time, and the routing-collapse check at the best epoch, from the Kaggle log or Weights & Biases; add the validation curves exported from Weights & Biases as `report/figures/task3_training_curves.png` if space allows]

[TODO: paragraph interpreting Table XIV and Fig. 12: which design is best on which corruption; whether the identity branch removes the ≈ 25 dB ceiling of Task 1 on clean and mildly corrupted inputs; the paired per-entry win rate and mean PSNR gain of the soft mixture over hard predicted routing, overall and split by whether the Task 2 classifier was right or wrong (`outputs/task3/tables/comparison_paired.csv`), remembering the bottleneck confound of Section V]

1) *Gating analysis*: [TODO: paragraph interpreting Figures 13–15 and Table XV: how routing weight varies with severity (are mild corruptions routed more softly?), whether any expert became inactive or dominates unrelated inputs (collapse verdict and mean normalised entropy from `outputs/task3/tables/routing_entropy.tex`), what the selected τ implies about where the system sits between hard and uniform routing, and whether the balance weight was large enough to matter. Expectation recorded before the run: because the gate starts from a near-perfect classifier, routing on the standard test set is likely to be almost one-hot, and the two-corruption experiment is where soft routing can differ]

C. Failure analysis

[TODO: discussion of Figure 16 (also `outputs/task3/figures/failures.png`): the worst mis-routed entries, whether soft routing recovers cases where the Task 2 classifier was confidently wrong, and cases where blending made the result worse than hard routing would have. Then the optional two-corruption experiment (`manifests/pets_mixed_manifest.json`; Table XVI, restoration in `outputs/task3/mixed/mixed_summary.tex`, examples in `outputs/task3/mixed/mixed_examples.png`), clearly labelled as beyond the brief’s corruption definitions: mean weights per combination, the share of entries whose two largest weights are the two corruptions present, and soft against hard routing on these inputs]

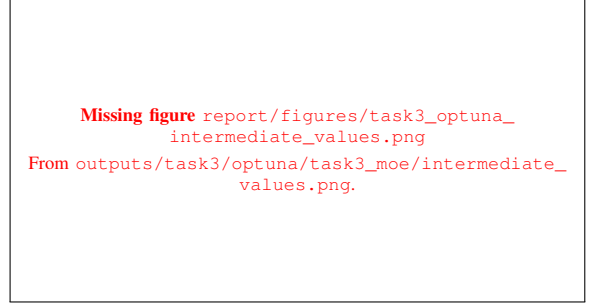
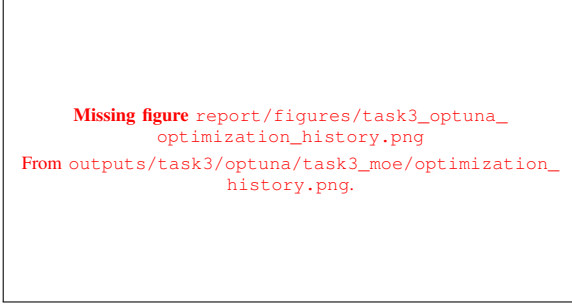


Fig. 11. Optuna study task3_moe (six trials on Kaggle): optimisation history (left) and validation score per epoch of every trial, including pruned ones (right).

TABLE XIV

CROSS-TASK COMPARISON ON IDENTICAL TEST ENTRIES: THE UNIVERSAL MODEL (TASK 1), HARD ROUTING WITH ORACLE AND PREDICTED ROUTES (TASK 2) AND THE SOFT MIXTURE (TASK 3). PSNR* EXCLUDES OUTPUTS IDENTICAL TO THE TARGET, SO THE CLEAN ROW OF HARD ROUTING IS OFTEN EMPTY; THE CLEAN ROW IS BEST COMPARED BY SSIM.

Input	Task 1 universal		Task 2 oracle		Task 2 predicted		Task 3 soft MoE	
	PSNR*	SSIM	PSNR*	SSIM	PSNR*	SSIM	PSNR*	SSIM
clean	-	-	-	-	-	-	-	-
salt	-	-	-	-	-	-	-	-
blur	-	-	-	-	-	-	-	-
occlusion	-	-	-	-	-	-	-	-

Missing table report/tables/task3_comparison_by_type.tex, copied from outputs/task3/tables/comparison_by_type.tex



Fig. 12. Cross-task comparison per input type and severity: PSNR* (left) and SSIM (right) of the universal model, hard routing (oracle and predicted) and the soft mixture on identical test entries.

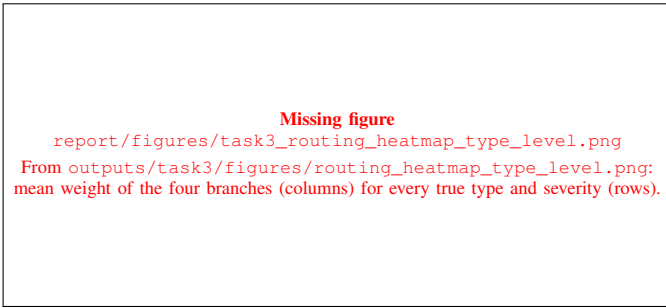


Fig. 13. Average expert weights for every true corruption type and severity level. Rows sum to one.

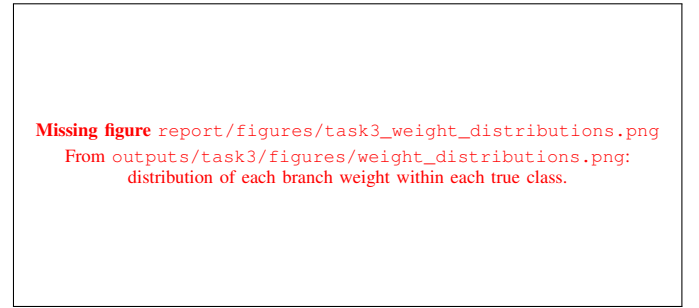


Fig. 14. Routing-weight distributions per true class.

VII. TASK 4: STYLE-CONDITIONED FACE-TO-SKETCH GENERATION

A. Methodology

1) *Generator*: The generator follows the pix2pix unet_128 layout [25]: seven 4×4 stride-2 convolutions

down to 1×1 with LeakyReLU(0.2), then six transposed convolutions with skip connections and a tanh output, with dropout after the three innermost decoder layers, following the convolution-normalisation-activation modules that pix2pix inherits from DCGAN [81] together with its $\mathcal{N}(0, 0.02)$ weight initialisation. It is worth noting the contrast with

Missing figure report/figures/task3_gating_dominant.png
From outputs/task3/figures/gating_dominant.png.

Missing figure report/figures/task3_gating_spread.png
From outputs/task3/figures/gating_spread.png.

Fig. 15. Inputs on which one expert dominates (left, lowest routing entropy per true class) and inputs on which the weights are spread across several branches (right, highest entropy), each with all four branch outputs and their weights, the mixed output and the target. Examples are selected by entropy rather than by hand, so they are reproducible.

TABLE XV

EXPERT ACTIVITY ON THE TEST SET: CLASS-BALANCED USAGE OF EACH BRANCH (1/4 UNDER BALANCED PERFECT ROUTING), MEAN WEIGHT ON ITS OWN CLASS AND ON THE OTHER CLASSES, THE LARGEST WEIGHT ON ANY OTHER CLASS, AND HOW OFTEN IT DOMINATES OTHER CLASSES.

Branch	Usage	Own	Others	Max other	on
identity	—	—	—	—	—

Missing table report/tables/task3_expert_activity.tex, copied from
outputs/task3/tables/expert_activity.tex

TABLE XVI

BEYOND THE BRIEF: TWO CORRUPTIONS PER IMAGE (MEDIUM SEVERITY). MEAN MIXTURE WEIGHTS PER COMBINATION AND THE SHARE OF ENTRIES WHOSE TWO LARGEST WEIGHTS ARE EXACTLY THE TWO CORRUPTIONS PRESENT.

Input	n	Identity	Salt	Blur	Occl.	Top-2 = parts
salt+blur	—	—	—	—	—	—

Missing table report/tables/task3_mixed_routing.tex, copied from
outputs/task3/mixed/mixed_routing.tex

Missing figure report/figures/task3_failures_misrouted.png
From outputs/task3/figures/failures_misrouted.png: the six
lowest-SSIM entries whose largest weight is not on the true branch.

Fig. 16. Task 3 failure cases: mis-routed entries with the lowest SSIM.

Section IV: the U-Net skips that are *forbidden* there, because they would defeat the bottleneck, are *required* here, because the goal is a translation that preserves the input’s spatial layout rather than a compressed representation. Architectural choices follow the objective.

Instance normalisation is used rather than batch normalisation. pix2pix’s batch normalisation with batch size one and test-batch statistics is instance normalisation in all but name [25], [82]; making it explicit means training and evaluation behave identically—so ONNX inference matches

training—and makes the network independent of the batch size, which Optuna tunes.

Style conditioning uses a single learned embedding of the three styles. At each of the six decoder layers, a conditional instance normalisation computes $\text{IN}(h) \cdot (1 + \gamma_l(e)) + \beta_l(e)$, where γ_l and β_l are linear projections of the shared embedding e . This is FiLM in the form of conditional instance normalisation [28], [27], [29]; projecting one shared embedding to every layer’s gains and biases follows BigGAN’s shared embedding [83], and modulating the decoder is what StarGAN v2 does for multi-domain translation [84], [31]. The encoder stays style-agnostic, since it only has to read the photograph.

The alternatives were not merely argued but measured in a controlled experiment: one synthetic photograph with three target sketches differing only in style, the same U-Net, two seeds, comparing the L_1 error under the true style against the error under a shifted style. A style-blind model can only reach the median of the three targets, around 0.054. Input concatenation scored 0.0558 with a gap of *exactly zero*—it learned nothing from the label, as the theory predicts: a spatially constant input map is only a conditional bias on the first layer, and the instance normalisation of the next layer subtracts per-channel offsets, so the signal is normalised away [29]. Injecting at the 1×1 bottleneck gave a gap of 0.0026, limited because the U-Net skips let the decoder bypass the bottleneck entirely. Decoder FiLM gave 0.0452 against 0.0555, a gap of 0.0104—four times the bottleneck variant—

and also modulating the encoder added nothing measurable (0.0108). Decoder FiLM was therefore chosen.

2) *Discriminator*: The discriminator is the pix2pix 70×70 PatchGAN on the concatenation of photograph and sketch, with three downsampling layers, instance normalisation and LeakyReLU(0.2), producing a 14×14 map of logits for 128×128 inputs. The 70×70 receptive field was verified by gradient propagation with the normalisations disabled. pix2pix found this patch size best among 1, 16, 70 and 286 pixels at 256×256 resolution: a 16×16 patch is enough for sharpness but tiles, and the full-image discriminator scored considerably worse [25]. At 128×128 a 70-pixel patch covers about half a face.

Style conditioning in the discriminator is by projection: the output layer is a shared 4×4 kernel plus a style-specific kernel generated from the discriminator’s own embedding, so each patch logit is $\text{logit}(p) = w \cdot \phi(p) + b + v(s) \cdot \phi(p)$. This is Miyato and Koyama’s projection discriminator, $f(x, y) = y^T V \phi(x) + \psi(\phi(x))$, applied per patch [33]; it follows from assuming that $p(y | x)$ is log-linear in the features, and on ImageNet it beat both concatenation and the AC-GAN auxiliary classifier [32]. Concatenation would face the same normalisation problem as in the generator. The photograph, being a genuinely spatial condition, is still supplied by concatenation as in pix2pix. Implementation detail: one convolution produces a logit map per style and a `gather` selects each row’s style, so only the used embedding rows receive gradients.

Two options are held in reserve should training prove unstable or style control weak: spectral normalisation of the discriminator [85] if it overpowers the generator, and mismatched-condition negatives—a real sketch presented with the wrong style and labelled fake, as in GAN-CLS [86]—if the source/style confound of Section III shows up as weak style control.

3) *Objective and training*: The generator objective is adversarial plus reconstruction,

$$\mathcal{L}_G = \mathcal{L}_{\text{adv}} + \lambda_{L_1} \mathcal{L}_{L_1}(y, G(x, s)), \quad (9)$$

with binary cross-entropy with logits for the adversarial terms. pix2pix uses $\lambda_{L_1} = 100$ and reports that L_1 alone gives blurry results while the conditional GAN alone gives sharp results with artefacts, the combination reducing them [25]; 100 is the starting point here and λ_{L_1} is searched. Following pix2pix, Adam [87] is used with $\beta = (0.5, 0.999)$ and the discriminator objective is halved to slow it relative to the generator; separate learning rates for the two networks are standard [88]. All spatial augmentation is applied identically to both members of a pair, as required to preserve pixel correspondence. Unlike pix2pix, dropout is active only in training mode: the application needs deterministic sketches and exact ONNX parity, so inference runs in evaluation mode.

4) *Optuna search design*: The study `task4_cgan` searches the seven quantities the brief names (Table XVII). The generator and discriminator share one base channel count and one style-embedding dimension, which keeps their capacities balanced and the search space small. Trial 0 is the pix2pix

TABLE XVII
OPTUNA SEARCH SPACE FOR TASK 4 (STUDY `TASK4_CGAN`, 18 TRIALS OF 12 EPOCHS ON KAGGLE), THE REFERENCE TRIAL 0, AND THE SELECTED CONFIGURATION.

Parameter	Range	Scale	Trial 0	Selected
η_G	5×10^{-5} – 5×10^{-4}	log	2×10^{-4}	–
η_D	5×10^{-5} – 5×10^{-4}	log	2×10^{-4}	–
Batch size	{4, 8, 16}	cat.	8	–
Base channels	{32, 48, 64}	cat.	64	–
Dropout	0.0–0.5	uniform	0.5	–
Style dim.	{8, 16, 32}	cat.	16	–
λ_{L_1}	25–200	log	100	–

[TODO: fill the Selected column from `outputs/task4/best_config.yaml`; state the best trial, its validation sketch score and the numbers of completed and pruned trials (`outputs/task4/optuna/task4_cgan/trials.csv`)]

and brief reference point. The objective is the best validation *sketch score*, $0.5 \text{ SSIM} + 0.5 (1 - L_1)$ on the validation split only, which involves neither λ_{L_1} nor the discriminator, so trials with different loss weights remain comparable; the sharpness diagnostic introduced below is recorded for every trial so that the blur a large λ_{L_1} induces stays visible. TPE is seeded at 42 and the median pruner waits for four trials and three epochs.

The final run trains the selected configuration from scratch for **200 epochs**—the pix2pix setting for datasets of this size [25]—with the pix2pix schedule: a constant learning rate for the first half, then linear decay to zero. The search ran **18 trials of 12 epochs** (35-minute cap). A first, smaller search (10 trials of 8 epochs) had selected a discriminator learning rate $4.5\times$ below the generator’s, and its 120-epoch model drew soft, faint strokes—a symptom of a discriminator too weak to penalise blur. The discriminator learning-rate range was therefore narrowed to start at 1.5×10^{-4} and the task was retrained; only the second run is reported.

The ranges are an octave either side of pix2pix’s 2×10^{-4} for both learning rates, searched separately because their ratio sets the balance between the two networks; batch sizes giving 224, 112 or 56 updates per epoch on the 899 training pairs; base widths giving 10.5, 23.6 or 41.9M generator parameters; a dropout range that includes pix2pix’s 0.5, its generator’s only noise source, and zero; and a λ_{L_1} range centred on the brief’s 100.

B. Results

[TODO: final 120-epoch run on Kaggle: best epoch, validation sketch score and epoch time (Kaggle log or Weights & Biases), and whether the mixed-precision losses stayed finite throughout]

What these metrics can and cannot support. SSIM [44] is paired, interpretable and the metric the FS2K benchmark itself reports, alongside the sketch-specific SCOOT measure [89], [35]; since SCOOT is not implemented here, the numbers above are *not* directly comparable with published FS2K benchmark results, and no such comparison is claimed. L_1 and PSNR measure pixel fidelity to the artist’s drawing, and the edge ratio measures directly the blur that an L_1 term

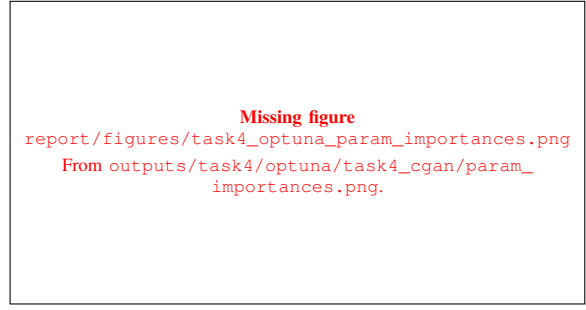
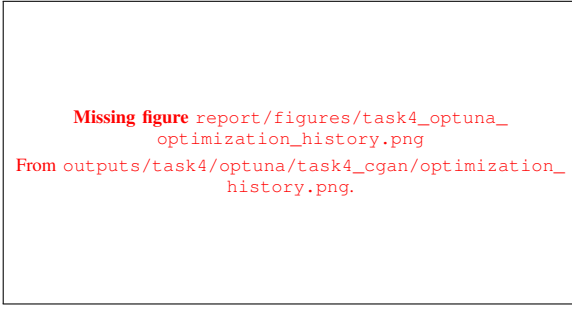


Fig. 17. Optuna study task4_cgan (ten trials on Kaggle): optimisation history (left) and parameter importances (right).

TABLE XVIII

TASK 4 RESULTS ON THE OFFICIAL FS2K TEST SET (1,046 PAIRS), PER STYLE: L_1 (ON $[0, 1]$), SSIM, PSNR (DB), EDGE RATIO (GENERATED EDGE ENERGY DIVIDED BY THE GROUND TRUTH’S; 1 = AS SHARP AS THE ARTIST) AND SKETCH SCORE. NOTE THE SUPPORT: STYLE 3 HAS ONLY 46 TEST PAIRS.

Style	n	L_1	SSIM	PSNR	Edge ratio	Score
Style 1	619	—	—	—	—	—
Style 2	381	—	—	—	—	—
Style 3	46	—	—	—	—	—

Missing table report/tables/task4_by_style.tex, copied from outputs/task4/tables/by_style.tex

TABLE XIX

TASK 4 TEST METRICS PER PHOTOGRAPH SOURCE AND STYLE. THE PHOTO3/STYLE 1 COMBINATION NEVER OCCURS IN THE TRAINING SPLIT (SECTION III).

Source	Style	n	L_1	SSIM	PSNR	Edge ratio
photo3	Style 1	179	—	—	—	—

Missing table report/tables/task4_by_source_style.tex, copied from outputs/task4/tables/by_source_style.tex

induces, which a pixel metric rewards rather than penalises. Two perceptual metrics were considered and deliberately *not* used. FID [88] is biased at finite sample sizes, and Chong and Forsyth [90] show that the bias depends on the model being evaluated, so one model can beat another purely through a smaller bias—an effect that dominates at the $n = 46$ of style 3; Parmar et al. [91] add that resizing and compression choices alone induce large FID variations. KID [92] would be better behaved at small n , but like FID it requires additional packages and ImageNet-trained weights downloaded at run time, which the project did not add. LPIPS [93] is paired and well defined on 46 images, but its perceptual calibration comes from patches of natural photographs, not from line drawings, so its judgement of sketch quality is not established. More generally, GAN metrics disagree [94], [95], so small differences should not be over-read, and visual evidence carries more weight here than in Tasks 1–3.

[TODO: paragraph reporting the style-sensitivity check (style_sensitivity in outputs/task4/eval/summary.json: test L_1 under the true style against the mean under the two wrong styles, and their gap) and what it says about how much the model actually uses the condition,

as opposed to inferring the style from the photograph’s appearance—the confound identified in Section III; then interpret Tables XVIII and XIX, including the seen/unseen split (by_seen_combination in the same file)]

C. Failure analysis

[TODO: failure cases from Figure 21 and outputs/task4/eval/failure_cases.csv: photographs far from the FS2K framing (off-centre or small faces), the unseen (source, style) combinations in the test set, the grey-paper offset affecting the 179 photo3 style-1 pairs, and any style for which the condition is weakly obeyed]

VIII. EXPERIMENTAL SETUP

Hardware. All models are trained with PyTorch [96] on NVIDIA T4 GPUs [97] with mixed precision [51] (fp16 autocast with gradient scaling, since the T4 has no bf16 support), in two environments. Task 1 (search, final training, evaluation and export) and both Task 2 searches ran on a free-tier Google Colab [98] instance with one T4. On the deadline day the free Colab GPU quota was exhausted, so the remaining work was moved to a Kaggle notebook with two T4 GPUs. A first Kaggle run produced working models for every task, but inspection showed over-smoothed restorations and faint sketches, so *all final models were retrained* in a second Kaggle run, which is the only one reported. A single runner, scripts/kaggle_run.py, prepares both datasets and drives three jobs in parallel: GPU 0 trains and evaluates the three Task 1 variants and writes the skip-ablation table; GPU 1 runs Task 4 and, alongside it, Task 2 (classifier and the three skip specialists) followed by Task 3, which is built from the new Task 2 models. Every step writes a completion marker, so the runner can simply be restarted after an interruption. The final models use the hyperparameters selected by the Optuna studies (the Task 1 and Task 2 studies ran on Colab and are committed as configs/task1.yaml and configs/task2_*.yaml); the deadline forced a shorter Optuna budget for Task 3 (Table XX).

Resumable runs. Because free-tier sessions disconnect without warning, every component is built to resume: training writes a complete checkpoint every epoch (model, optimiser, scheduler, gradient scaler, early-stopping state, history and the Weights & Biases run id) and --resume continues from

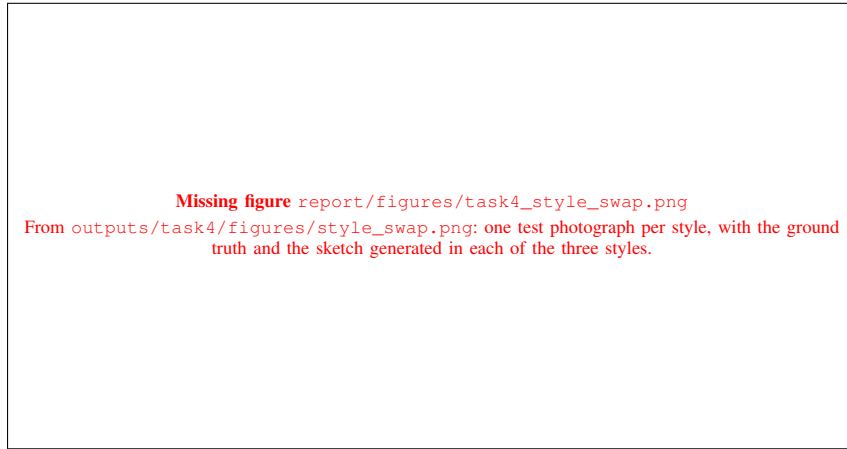


Fig. 18. The same photograph rendered in all three styles, next to the ground-truth sketch. The style condition is the only input that differs across the three generated columns.

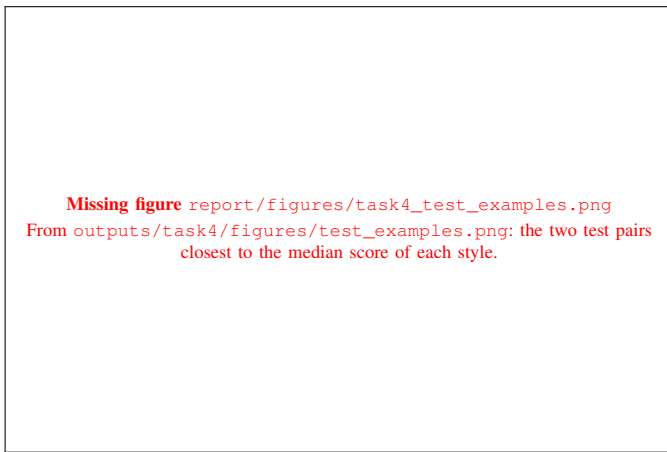


Fig. 19. Typical Task 4 test results (median sketch score per style): photograph, ground truth and generated sketch.

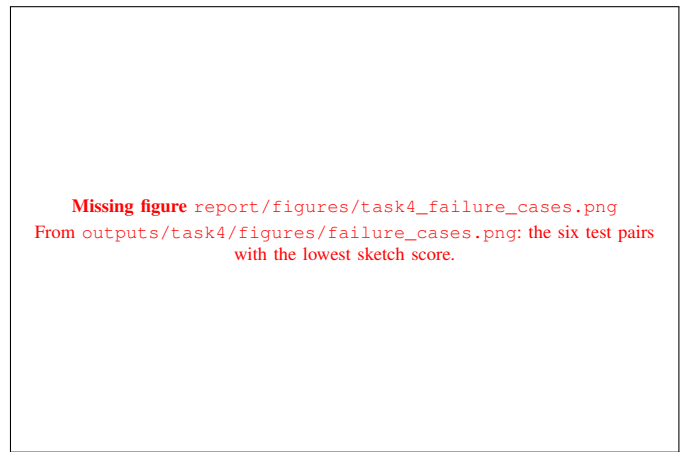


Fig. 21. Task 4 failure cases (lowest sketch score); automatic notes on each case are written to `outputs/task4/eval/failure_cases.csv`.

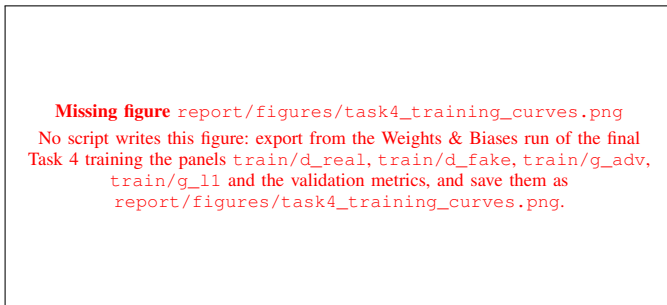


Fig. 20. Task 4 training curves: discriminator real and fake losses, generator adversarial and reconstruction losses, and validation measurements, each logged separately as the brief requires.

it with the stored configuration treated as authoritative, so a resumed run cannot silently change its schedule. The best checkpoint is written before the last one, so a crash between the two can never leave the best behind. Data order and runtime corruptions depend only on the seed and the epoch index, so a resumed run sees the same data as an uninterrupted one; a test interrupts a run mid-epoch and asserts that the

resumed weights match the uninterrupted ones exactly.

Hyperparameter optimisation uses Optuna [56], [99]. Its define-by-run interface allows the search space to be built dynamically, which is what lets conditional parameters (such as a latent shape that implies a depth) be expressed directly. All studies use the TPE sampler [55], [100] seeded at 42, which fits one density to the parameters of the best trials and another to the rest and maximises their ratio; random search would be the weaker baseline [101]. Pruning uses the median stopping rule [57], which stops a trial whose best intermediate value is worse than the median of completed trials at the same step and which Vizier reports gives a two- to three-fold speedup while still finding the best trial. Successive halving and Hyperband [102], [103], [104] were considered and not used: with 20–40 trials on one T4, their aggressive early stopping would spend most of the budget on very short runs whose ranking is unreliable for GAN and mixture training, whereas the median rule is model-free and matches the fixed-epoch validation schedule. Task 3 adds the task-specific routing-collapse criterion of Section VI, which

TABLE XX

OPTUNA BUDGETS AND FINAL TRAINING RUNS. C/P: COMPLETED AND PRUNED TRIALS; TIME: SUM OF TRIAL DURATIONS. THE TASK 3 BUDGET WAS CUT ON KAGGLE; THE ORIGINAL PLAN WAS 20 TRIALS OF 1+4 EPOCHS AND A 2+20-EPOCH FINAL RUN.

Study	GPU	Trials $\times$ ep.	C/P	Time	Final run
T1 UDAE	Colab	33 (of 40) $\times$ 15	15/18	89.6 min	3 $\times$ 60 ep., Kaggle
T2 classifier	Colab	30 $\times$ 12	5/25	15.6 min	60 ep., Kaggle
T2 specialists	Colab	20 $\times$ 6 ($\times$ 3)	12/8	61.5 min	3 $\times$ 40 ep., Kaggle
T3 soft MoE	Kaggle	6 $\times$ (1+3)	—	—	2+10 ep.
T4 cGAN	Kaggle	18 $\times$ 12	—	—	200 ep.

[TODO: fill the Task 3 and Task 4 rows and the Kaggle final-run times from outputs/task3/optuna/task3_moe/summary.json, outputs/task4/optuna/task4_cgan/summary.json and the Kaggle logs]

no generic pruner can express. Each study’s SQLite database lives on local disk and a consistent snapshot is copied to persistent storage (Google Drive on Colab, the notebook’s working directory on Kaggle) after every finished trial, so a search survives a disconnect and only the remaining trials are run. Table XX lists what each study actually ran.

Experiment tracking uses Weights & Biases [105], in the project `genai-a1` (<https://wandb.ai/mustafafaraz87-fast-nuces/genai-a1>). Every Optuna trial is its own run, grouped per study, with its parameters in the configuration and its per-epoch validation metrics logged; final runs additionally log per-condition metrics, sample image grids at fixed intervals using *fixed* validation indices so that progress is comparable across epochs, and the best checkpoint as a model artefact. The Colab and Kaggle runs log to the same project. [TODO: make the Weights & Biases project public (or link a W&B report) before submission]

IX. APPLICATION ARCHITECTURE

The four tasks are served by one application, *GenAI Studio*, with four workspaces named as in the brief: **Universal Restoration** (Task 1), **Hard-Routed Restoration** (Task 2), **Soft Mixture-of-Experts Restoration** (Task 3) and **Face-to-Sketch Generator** (Task 4). Figure 23 shows its structure.

A. Interface design

The interface was designed in Google Stitch [106] before any frontend code was written, as the brief requires; Fig. 22 reproduces the original Stitch screens, and the exported HTML is kept in `design/stitch/`. The design defines a four-tab shell over a 12-column grid: a narrow input card (upload dropzone, sample images, *Corruption* and *Severity* segmented controls and the action button) and a wide results column with *Original*, *Model input* and *Restored output* panels, a details card with a download control and an error map. Each workspace adds what its task exposes: a *Predicted/Oracle* routing toggle and classifier probability bars for hard routing; four expert weights, a “main contributor” and the four branch outputs for the mixture; and a webcam button and a *Style 1/2/3* selector for the sketch generator. The implementation keeps the design’s Tailwind configuration (colour roles, radii,

TABLE XXI

BACKEND API (ALL PATHS UNDER `/API`; EVERY REQUEST IS MULTIPART FORM DATA AND EVERY IMAGE IN A RESPONSE A PNG DATA URL).

Endpoint	Purpose
GET <code>/health</code>	status, uptime, runtime versions, models and model cards, ready workspaces
GET <code>/options</code>	corruption types, levels, training ranges, styles, upload limits
GET <code>/samples</code>	bundled pet photographs from the official test split
POST <code>/corrupt</code>	runtime corruption preview (type, level or custom parameters, seed)
POST <code>/restore/universal</code>	Task 1, <code>udae.onnx</code>
POST <code>/restore/hard</code>	Task 2, classifier + one specialist or identity; predicted or oracle routing
POST <code>/restore/moe</code>	Task 3, <code>moe.onnx</code> : output, four weights, four branch outputs
POST <code>/sketch</code>	Task 4, <code>generator.onnx</code> , style 1, 2 or 3

spacing and type scale) unchanged, but every number in the mock-ups—GPU names, latencies, LPIPS scores—was a Stitch placeholder and was replaced by values returned by the API or removed: the application shows only real data.

B. Backend

The backend is a FastAPI application [107] that runs every model through ONNX Runtime [108] on the CPU execution provider, so it needs no GPU and does not import PyTorch at all (a test asserts it). Its endpoints are listed in Table XXI. The health endpoint reports status, uptime, runtime versions, every model with its model card, and which workspaces are ready; the options endpoint returns the corruption types, fixed levels, permitted custom parameter ranges and style names, so the interface builds its controls from the server rather than duplicating constants. Crucially, the backend imports *the same* corruption and preprocessing code as training (`src/data/`), so an image passed through the application is degraded and prepared exactly as it would have been during training, and the corrupted input is rounded to 8 bits so that the image returned as “Model input” is exactly what the model saw.

Uploads are validated by decoding rather than by declared content type: JPEG, PNG, WEBP and BMP are accepted up to 10 MB and 50 megapixels, which rejects decompression bombs; EXIF rotation is applied and greyscale, palette and transparent images are converted to RGB. Parameters outside the training ranges are refused with an explanatory message rather than silently clamped, so the evaluator cannot unknowingly test the models outside the distribution they were trained on. Requests may supply either an uploaded file or a bundled sample, and either a fixed severity level or custom parameters, never both. When the server applies the corruption itself, the response also contains PSNR and SSIM of the model input and of the output against the clean original, with SSIM implemented in NumPy to the training definition (agreement with the PyTorch version within 10^{-5} is tested), and every response reports the inference and total server time—for hard

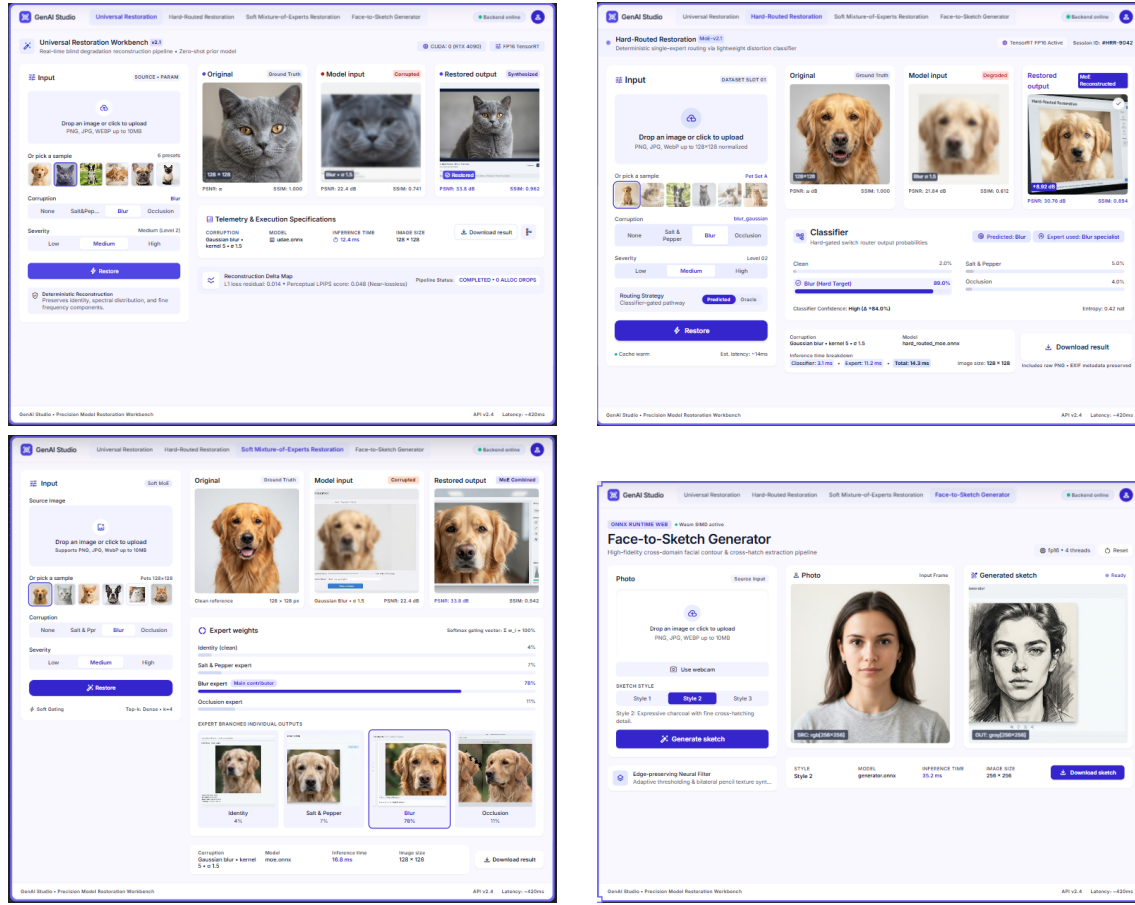


Fig. 22. The original Google Stitch designs of the four workspaces, as generated in Stitch before implementation (evidence required by the brief): Universal Restoration (top left), Hard-Routed Restoration (top right), Soft Mixture-of-Experts Restoration (bottom left) and Face-to-Sketch Generator (bottom right). The numbers and model names in these mock-ups are Stitch placeholders; the implemented application replaces them with values returned by the backend.

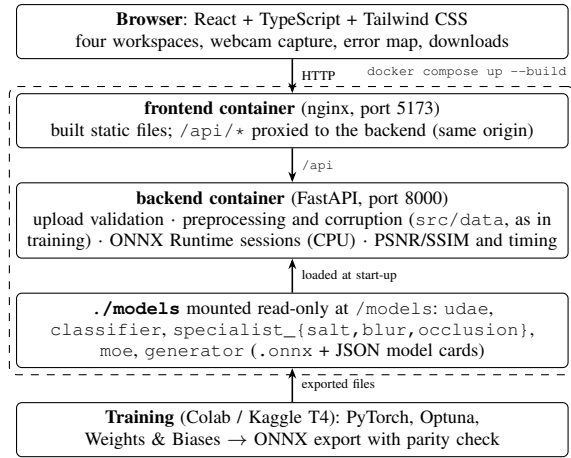


Fig. 23. Application architecture. One command starts both containers; the backend reuses the training code’s corruption and preprocessing functions, so the application degrades and prepares images exactly as training did.

routing split into classifier and expert time. A missing model file disables only the requests that need it, with a message naming the file.

C. Frontend and deployment

The frontend is a React [109] and TypeScript application styled with Tailwind CSS [110] and built with Vite. It implements the four Stitch workspaces against the API, each at its own URL and kept mounted when hidden, so inputs and results survive switching tabs. Each restoration workspace accepts an upload or a sample and offers runtime corruption with a fixed level or custom parameters and a reproducible seed, a corruption preview, PSNR and SSIM under the image panels, an absolute error map computed in the browser and a download of the result; the hard-routing workspace adds the classifier probabilities, the predicted and true class, the expert used and the classifier/expert timing, and the mixture workspace the four expert weights and branch outputs. The Face-to-Sketch Generator accepts an upload or a webcam capture and a style, shows photograph and sketch side by side and offers the sketch for download. Fonts and icons are self-hosted from npm packages and Tailwind is compiled at build time, so the application needs no external CDN.

Frontend and backend each run in their own container and start together with one documented command, `docker compose up --build` [111]. The application is then

served at <http://localhost:5173>; the frontend container (nginx) serves the built files and forwards `/api` to the backend container, so the browser talks to a single origin and no CORS configuration is needed. The backend publishes port 8000 with interactive API documentation, runs as a non-root user with a health check, and the frontend starts once that check passes. The trained ONNX models are not baked into the image: the `./models` folder is mounted read-only, so updated models need only a restart, and without them the backend still starts, reports itself as degraded and the interface names the missing files.

D. ONNX export and parity

Seven ONNX [112], [113] files are exported, at opset 17 in float32 with a dynamic batch axis: the universal autoencoder, the classifier, the three specialists, the complete mixture pipeline as a single graph (gate, temperature constant, softmax, identity branch, three experts and the weighted sum, emitting the output, the four weights and all four branch outputs) and the sketch generator; the discriminator is a training component only. The TorchScript-based exporter is used explicitly, because the newer default exporter of recent PyTorch versions targets opset 18 and can emit invalid `Split` nodes. Every export is verified against its PyTorch original in evaluation mode on at least 32 *real* test inputs spanning all conditions and severities—not random noise, so that the gate sees realistic logits—and the maximum absolute difference must stay below 10^{-4} , or the file is rejected and the script fails. [TODO: measured parity of all exported models (from the model cards)] Each model is accompanied by a JSON model card recording its configuration, input and output specification, preprocessing, test metrics, hyperparameters, the measured parity, the Weights & Biases run URL, the git commit and the export time; the backend shows these cards in the application’s system-information panel. [TODO: measured parity of the classifier, the three specialists, the mixture and the generator, from the model cards of the Kaggle run]

X. LIMITATIONS

Severity coverage. All three test “high” levels sit at the upper edge of the training ranges, and no training blur ever reaches the strength of the (7,2.5) test configuration (Section III). Results at the high level therefore measure extrapolation to the boundary of the training distribution as much as restoration ability; for Task 1 this shows as a 0.8 dB drop from medium to high blur and a steady decline for occlusion, while salt-and-pepper is unaffected.

Search fidelity and short-trial bias. Hyperparameters were selected from short trials—15 epochs for Task 1, 12 for the Task 2 classifier, 6 per specialist, 4 for Task 3 and 8 for Task 4—and applied to runs three to fifteen times longer. Short budgets favour configurations that learn quickly, and the evidence is visible: both restoration searches chose the smallest batch size, which gives the most optimiser steps per epoch, and Task 1’s selected configuration gained 0.076 in validation score between its 15-epoch trial and the 60-epoch

run. The selected configurations are good regions of the search space found under a budget, not proven optima; Task 1’s best trial was also its last, so more trials could still have helped.

Reduced budgets for Tasks 3 and 4. The free Colab GPU quota ran out on the deadline day, and Tasks 3 and 4 were trained on Kaggle with budgets cut to fit: six Optuna trials of one warm-up and three joint epochs and a final run of 2+10 epochs for Task 3 (planned: 20 trials, 2+20 epochs), and ten trials of eight epochs for Task 4 (planned: 20 of ten). With ten random start-up trials configured for TPE, neither study reached the model-based phase of the sampler, so both are small random searches around the brief’s reference configuration. Their hyperparameters are therefore less well optimised than those of Tasks 1 and 2, and Task 3’s shorter joint fine-tuning may understate what soft routing can achieve.

The Task 1 versus Task 2 bottleneck confound. The specialists’ search offered 16×16 latents and selected one of 8,192 values, twice the 4,096 of the universal model, whose range was capped to keep a meaningful bottleneck. Part of any advantage of hard routing over the universal model may therefore come from the larger latent rather than from specialisation; a controlled comparison would retrain the universal model with the same latent shape.

Missing ablation. The limited-skip ablation of Task 1 is implemented but was not trained within the GPU budget, so the leak-versus-detail trade-off of skip connections is argued and bounded by the two diagnostics rather than measured.

Mixed precision for the GAN. Task 4 trains with fp16 autocast and separate gradient scalers for the generator and the discriminator. Its numerical stability could not be verified before the Kaggle run, since the development machine had no GPU; a non-finite loss aborts the run (or prunes the trial) rather than producing silent NaNs, and full precision remains available as a fallback. [TODO: confirm from the Kaggle log that every Task 4 trial and the final run kept finite losses, or state which trials were pruned for non-finite losses]

Scale. With 2,944 training images at 128×128 and models of a few million parameters, none of these systems is comparable with published all-in-one restoration models [2], [3], which are trained on orders of magnitude more data. No such comparison is made.

The mixture objective. As Section VI notes, training the blend on its reconstruction error is the cooperative formulation that Jacobs et al. identify as discouraging specialisation [14]. The pre-trained experts and the cross-entropy term counteract it, but the system is not an independent demonstration that soft routing induces specialisation; the specialisation was supplied by Task 2.

Balance regulariser blind spot. Equation (7) sees only batch-mean weights and cannot detect a gate that routes each class confidently to the wrong branch. The routing matrix and the cross-entropy term cover that case, but the regulariser alone does not.

FS2K confounds and sample sizes. Style is confounded with photograph source in the official FS2K training split—styles 1 and 2 come only from `photo1`, style 3 only from `photo2`



Fig. 24. The four workspaces of the running application with the trained models.

and photo3—so part of what looks like style control may be source recognition; the test set contains only 46 style-3 pairs, which makes distributional metrics unusable for that style and widens the error bars on every per-style number; and the 179 photo3 style-1 test pairs carry the grey-paper background offset (Section III).

Metrics. PSNR correlates with subjective quality only within a fixed content set [114], and SSIM and PSNR are shallow functions [93]. Task 4 reports no FID and no LPIPS: FID is biased in a model-dependent way at small sample sizes [90] and would be meaningless on the 46 style-3 pairs, LPIPS is calibrated on natural photographs rather than line drawings, and both need extra packages and downloaded ImageNet weights that the project did not add. Sketch quality is therefore judged by L_1 , PSNR, SSIM, the edge ratio and visual evidence, which measure fidelity to one artist’s drawing rather than perceptual quality. Conclusions are drawn from several metrics together with visual evidence, never from a single number.

Deployment. ONNX inference runs on CPU in the application container, so reported inference times reflect the evaluation machine and are not comparable with GPU timings.

XI. CONCLUSION

[TODO: conclusion, to be written once results exist. It should state: which of the three restoration designs performed best overall and on which corruptions; whether explicit routing (Task 2) justified its extra complexity over one shared model (Task 1); whether making the routing soft and differentiable (Task 3) recovered the classifier errors it was intended to, and at what cost; whether the Task 4 generator genuinely uses

the style condition rather than inferring style from the photograph; and which of the research-driven design decisions (pure bottleneck, L1/SSIM mixture, resize-convolution, balance regulariser, decoder FiLM with a projection discriminator) were confirmed by the experiments and which were not.]

APPENDIX USE OF AI ASSISTANCE

AI tools were used extensively during this work, as the brief permits, and Table XXII lists them. The main one was Claude Code (Anthropic, with Claude Opus models), used as a coordinating coding assistant that split the work into parallel sub-agents, each with a narrow assignment and its own tests: one for research and citation verification, one for the data pipeline and the shared training utilities, one for each task’s training, evaluation and export code, and one each for the FastAPI backend, the React frontend, the Docker setup and the drafting of this report. The interface was designed with Google Stitch.

Nothing generated was accepted on trust. Every component ships with unit and smoke tests—111 Python tests for the data pipeline, models and the four tasks, 97 backend tests and 54 frontend tests—and the coordinating session re-ran every sub-agent’s tests independently before anything was committed. Every ONNX export must pass its parity check. Every one of the 114 references was checked against Crossref, arXiv or the publisher’s or proceedings page, with the verifying URL recorded next to its BibTeX entry, and quoted claims were spot-checked independently. Verification found and corrected real errors, five of which illustrate the process:

- 1) *Blur severity mislabelled.* Blur severity was first assigned from σ alone. Measuring the strength of the kernel actually applied (Eq. 1) showed that a wide Gaussian truncated to three taps is milder than the medium test level although its σ is larger; relabelling by measured strength corrected 86 of the 184 validation blur labels.
- 2) *SSIM convention.* A cross-check against `torchmetrics` found a small disagreement (about 3×10^{-5}). The cause is a convention, not a bug: `torchmetrics` pads the image and averages over all windows, whereas the implementation here follows Wang et al.'s reference and averages over the valid region only. The difference is documented, and the test now asserts agreement with `torchmetrics`' SSIM map cropped to the valid region.
- 3) *SQLite connection leak on Windows.* In the code that snapshots each Optuna study to persistent storage, a `sqlite3` connection used as a context manager commits but is not closed, which on Windows kept the snapshot file locked; it is now closed explicitly.
- 4) *A quick check that contaminated real checkpoints.* The first Task 2 classifier evaluation reported test accuracy 0.300, macro-recall exactly 0.250 and validation macro-F1 0.1000—one class predicted for every input. Checking these numbers against what a trained model must produce exposed the cause: a `--smoke` run had saved a finished tiny model into the real checkpoint folder, so the real run, started with `--resume`, found “nothing to do” and evaluation silently used the tiny model. Real runs now ignore smoke checkpoints, evaluation and export refuse them, and a regression test covers the case. This is the clearest example of generated code that passed its own tests yet failed in the real workflow.
- 5) *Zhao et al.'s α .* Their recommended $\alpha = 0.84$ looks like support for the brief's $\alpha = 0.8$, but checking the paper showed that it weights the MS-SSIM term, not L_1 , so it corresponds to $\alpha \approx 0.16$ here. The search range was set to contain both values, and the selected 0.285 is interpreted with the correct parameterisation (Section IV).

The author reviewed all submitted code, text and figures, ran and checked the experiments, and is responsible for the correctness of the complete submission.

AVAILABILITY

The complete implementation—data preparation, training, Optuna search, evaluation and ONNX export scripts, backend, frontend, Dockerfiles and Docker Compose configuration—is available at <https://github.com/fsdev87/generative-ai-A1>; the application starts with `docker compose up --build` once the ONNX models are placed in `models/` [TODO: download link of the trained ONNX models]. Experiment tracking records are in the Weights & Biases project <https://wandb.ai/mustafafaraz87-fast-nuces/genai-a1>. The demonstration video is at [TODO: YouTube link].

REFERENCES

- [1] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, “Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising,” *IEEE Transactions on Image Processing*, vol. 26, no. 7, pp. 3142–3155, 2017.
- [2] B. Li, X. Liu, P. Hu, Z. Wu, J. Lv, and X. Peng, “All-in-one image restoration for unknown corruption,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 17431–17441.
- [3] V. Potlapalli, S. W. Zamir, S. Khan, and F. Shahbaz Khan, “PromptIR: Prompting for all-in-one image restoration,” in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 71275–71293.
- [4] K. Yu, C. Dong, L. Lin, and C. C. Loy, “Crafting a toolchain for image restoration by deep reinforcement learning,” in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2443–2452.
- [5] K. Yu, X. Wang, C. Dong, X. Tang, and C. C. Loy, “Path-restore: Learning network path selection for image restoration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 10, pp. 7078–7092, 2022.
- [6] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, “Extracting and composing robust features with denoising autoencoders,” in *Proceedings of the 25th International Conference on Machine Learning (ICML)*. ACM, 2008, pp. 1096–1103.
- [7] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol, “Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion,” *Journal of Machine Learning Research*, vol. 11, no. 110, pp. 3371–3408, 2010.
- [8] G. E. Hinton and R. R. Salakhutdinov, “Reducing the dimensionality of data with neural networks,” *Science*, vol. 313, no. 5786, pp. 504–507, 2006.
- [9] J. Masci, U. Meier, D. Cireřan, and J. Schmidhuber, “Stacked convolutional auto-encoders for hierarchical feature extraction,” in *Artificial Neural Networks and Machine Learning – ICANN 2011*, ser. Lecture Notes in Computer Science. Springer, 2011, pp. 52–59.
- [10] J. Xie, L. Xu, and E. Chen, “Image denoising and inpainting with deep neural networks,” in *Advances in Neural Information Processing Systems*, vol. 25. Curran Associates, Inc., 2012.
- [11] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, ser. Lecture Notes in Computer Science. Springer, 2015, pp. 234–241.
- [12] X. Mao, C. Shen, and Y.-B. Yang, “Image restoration using very deep convolutional encoder-decoder networks with symmetric skip connections,” in *Advances in Neural Information Processing Systems*, vol. 29. Curran Associates, Inc., 2016, arXiv:1603.09056.
- [13] D. Pathak, P. Krähenbühl, J. Donahue, T. Darrell, and A. A. Efros, “Context encoders: Feature learning by inpainting,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2536–2544.
- [14] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, “Adaptive mixtures of local experts,” *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [15] M. I. Jordan and R. A. Jacobs, “Hierarchical mixtures of experts and the EM algorithm,” *Neural Computation*, vol. 6, no. 2, pp. 181–214, 1994.
- [16] S. E. Yuksel, J. N. Wilson, and P. D. Gader, “Twenty years of mixture of experts,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 23, no. 8, pp. 1177–1193, 2012.
- [17] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *5th International Conference on Learning Representations (ICLR)*, 2017, arXiv:1701.06538.
- [18] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [19] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, “GShard: Scaling giant models with conditional computation and automatic sharding,” in *9th International Conference on Learning Representations (ICLR)*, 2021, arXiv:2006.16668.

TABLE XXII
USE OF AI ASSISTANCE: TOOLS, WHAT THEY WERE USED FOR, AND HOW THEIR OUTPUT WAS TESTED OR CORRECTED.

Tool	Used for	How the output was tested or corrected
Claude Code (Anthropic), coordinating session	Planning, splitting the work into parallel sub-agents, integration, debugging, commits	Each sub-agent’s tests re-run independently before every commit; the student reviewed diffs, ran the training on Colab and Kaggle and checked results against expectations (correction 4)
Claude Code sub-agent: research	Literature search, comparison of design alternatives, collection of the 114 references	Every citation checked against Crossref, arXiv or the proceedings page (URL kept in <code>references.bib</code>); quotes spot-checked independently; parameterisation errors corrected (5)
Claude Code sub-agents: data and shared code	Oxford-IIIT Pet split, runtime corruptions, manifests, balanced sampler, FS2K preparation; losses, metrics, checkpoints, Optuna and W&B helpers, ONNX export	44 tests (corruptions, FS2K, models, shared utilities), e.g. blur equal to <code>torchvision</code> within 10^{-5} , exact resume after interruption; corrections 1–3
Claude Code sub-agents: Tasks 1–4	Models, training, Optuna studies, evaluation, figures and ONNX export for each task	67 CPU smoke and unit tests (14, 14, 17 and 22 per task), ONNX parity at most 10^{-4} on real inputs, evaluation numbers sanity-checked (correction 4)
Claude Code sub-agents: backend, frontend, Docker	FastAPI service, React + Tailwind interface implementing the Stitch design, Dockerfiles and Compose file	97 backend tests (validation, errors, endpoints, metrics, missing models), 54 Vitest tests, end-to-end run under Docker Compose with placeholder models, and the exported Task 1 model checked through the real backend
Claude Code sub-agent: report	Drafting this report from the design notes and the experiment log	Every number traced to the experiment log or to the output files of the scripts; text reviewed by the student
Google Stitch	Visual design of the four workspaces (Fig. 22)	Design reviewed; its Tailwind configuration reused; placeholder numbers replaced by data from the API

- [20] J. Puigcerver, C. Riquelme, B. Mustafa, and N. Houlsby, “From sparse to soft mixtures of experts,” in *12th International Conference on Learning Representations (ICLR)*, 2024, arXiv:2308.00951.
- [21] E. Bengio, P.-L. Bacon, J. Pineau, and D. Precup, “Conditional computation in neural networks for faster models,” 2015.
- [22] D. Eigen, M. Ranzato, and I. Sutskever, “Learning factored representations in a deep mixture of experts,” 2013.
- [23] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Advances in Neural Information Processing Systems*, vol. 27. Curran Associates, Inc., 2014.
- [24] M. Mirza and S. Osindero, “Conditional generative adversarial nets,” 2014.
- [25] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, “Image-to-image translation with conditional adversarial networks,” in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 5967–5976, arXiv:1611.07004.
- [26] Y. Choi, M. Choi, M. Kim, J.-W. Ha, S. Kim, and J. Choo, “StarGAN: Unified generative adversarial networks for multi-domain image-to-image translation,” in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 8789–8797.
- [27] V. Dumoulin, J. Shlens, and M. Kudlur, “A learned representation for artistic style,” in *5th International Conference on Learning Representations (ICLR)*, 2017, arXiv:1610.07629.
- [28] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “FiLM: Visual reasoning with a general conditioning layer,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [29] V. Dumoulin, E. Perez, N. Schucher, F. Strub, H. de Vries, A. Courville, and Y. Bengio, “Feature-wise transformations,” *Distill*, vol. 3, no. 7, 2018.
- [30] H. de Vries, F. Strub, J. Mary, H. Larochelle, O. Pietquin, and A. Courville, “Modulating early visual processing by language,” in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017.
- [31] X. Huang and S. Belongie, “Arbitrary style transfer in real-time with adaptive instance normalization,” in *2017 IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 1510–1519.
- [32] A. Odena, C. Olah, and J. Shlens, “Conditional image synthesis with auxiliary classifier GANs,” in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 2642–2651.
- [33] T. Miyato and M. Koyama, “cGANs with projection discriminator,” in *6th International Conference on Learning Representations (ICLR)*, 2018, arXiv:1802.05637.
- [34] X. Wang and X. Tang, “Face photo-sketch synthesis and recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 31, no. 11, pp. 1955–1967, 2009.
- [35] D.-P. Fan, Z. Huang, P. Zheng, H. Liu, X. Qin, and L. Van Gool, “Facial-sketch synthesis: A new challenge,” *Machine Intelligence Research*, vol. 19, no. 4, pp. 257–287, 2022.
- [36] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, “Cats and dogs,” in *2012 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 3498–3505.
- [37] Visual Geometry Group, University of Oxford, “The Oxford-IIIT pet dataset,” <https://www.robots.ox.ac.uk/~vgg/data/pets/>, 2026, accessed 3 October 2026.
- [38] D.-P. Fan, “FS2K: Facial-sketch synthesis dataset,” <https://github.com/DengPingFan/FS2K>, 2026, accessed 3 October 2026.
- [39] M. Buda, A. Maki, and M. A. Mazurowski, “A systematic study of the class imbalance problem in convolutional neural networks,” *Neural Networks*, vol. 106, pp. 249–259, 2018.
- [40] D. Yi, Z. Lei, S. Liao, and S. Z. Li, “Learning face representation from scratch,” 2014.
- [41] A. Odena, V. Dumoulin, and C. Olah, “Deconvolution and checkerboard artifacts,” *Distill*, vol. 1, no. 10, 2016.
- [42] A. Aitken, C. Ledig, L. Theis, J. Caballero, Z. Wang, and W. Shi, “Checkerboard artifact free sub-pixel convolution: A note on sub-pixel convolution, resize convolution and convolution resize,” 2017.
- [43] W. Shi, J. Caballero, F. Huszár, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert, and Z. Wang, “Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1874–1883.
- [44] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [45] H. Zhao, O. Gallo, I. Frosio, and J. Kautz, “Loss functions for image restoration with neural networks,” *IEEE Transactions on Computational Imaging*, vol. 3, no. 1, pp. 47–57, 2017, arXiv:1511.08861.
- [46] Z. Wang, E. P. Simoncelli, and A. C. Bovik, “Multiscale structural similarity for image quality assessment,” in *The Thirty-Seventh Asilomar Conference on Signals, Systems and Computers*, 2003, pp. 1398–1402.
- [47] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual losses for real-time style transfer and super-resolution,” 2016.
- [48] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *7th International Conference on Learning Representations (ICLR)*, 2019, arXiv:1711.05101.

- [49] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, “Accurate, large minibatch SGD: Training ImageNet in 1 hour,” 2017.
- [50] I. Loshchilov and F. Hutter, “SGDR: Stochastic gradient descent with warm restarts,” in *5th International Conference on Learning Representations (ICLR)*, 2017, arXiv:1608.03983.
- [51] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, “Mixed precision training,” in *6th International Conference on Learning Representations (ICLR)*, 2018, arXiv:1710.03740.
- [52] J. Tompson, R. Goroshin, A. Jain, Y. LeCun, and C. Bregler, “Efficient object localization using convolutional networks,” in *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 648–656.
- [53] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [54] A. Hore and D. Ziou, “Image quality metrics: PSNR vs. SSIM,” in *2010 20th International Conference on Pattern Recognition (ICPR)*, 2010, pp. 2366–2369.
- [55] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyper-parameter optimization,” in *Advances in Neural Information Processing Systems*, vol. 24. Curran Associates, Inc., 2011.
- [56] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 2019, pp. 2623–2631.
- [57] D. Golovin, B. Solnik, S. Moitra, G. Kochanski, J. Karro, and D. Sculley, “Google Vizier: A service for black-box optimization,” in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2017, pp. 1487–1495.
- [58] H. Hwang and R. A. Haddad, “Adaptive median filters: New algorithms and results,” *IEEE Transactions on Image Processing*, vol. 4, no. 4, pp. 499–502, 1995.
- [59] R. H. Chan, C.-W. Ho, and M. Nikolova, “Salt-and-pepper noise removal by median-type noise detectors and detail-preserving regularization,” *IEEE Transactions on Image Processing*, vol. 14, no. 10, pp. 1479–1485, 2005.
- [60] W. H. Richardson, “Bayesian-based iterative method of image restoration,” *Journal of the Optical Society of America*, vol. 62, no. 1, pp. 55–59, 1972.
- [61] L. B. Lucy, “An iterative technique for the rectification of observed distributions,” *The Astronomical Journal*, vol. 79, pp. 745–754, 1974.
- [62] A. Telea, “An image inpainting technique based on the fast marching method,” *Journal of Graphics Tools*, vol. 9, no. 1, pp. 23–34, 2004.
- [63] M. Bertalmio, G. Sapiro, V. Caselles, and C. Ballester, “Image inpainting,” in *Proceedings of the 27th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, 2000, pp. 417–424.
- [64] M. Bertalmio, A. L. Bertozzi, and G. Sapiro, “Navier-stokes, fluid dynamics, and image and video inpainting,” in *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 1, 2001, pp. I–355–I–362.
- [65] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [66] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [67] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 37. PMLR, 2015, pp. 448–456.
- [68] T. He, Z. Zhang, H. Zhang, Z. Zhang, J. Xie, and M. Li, “Bag of tricks for image classification with convolutional neural networks,” in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 558–567.
- [69] M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks,” *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.
- [70] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 1321–1330.
- [71] J. L. Pech-Pacheco, G. Cristóbal, J. Chamorro-Martínez, and J. Fernández-Valdivia, “Diatom autofocusing in brightfield microscopy: A comparative study,” in *Proceedings 15th International Conference on Pattern Recognition (ICPR)*, vol. 3, 2000, pp. 314–317.
- [72] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
- [73] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” 2015.
- [74] E. Jang, S. Gu, and B. Poole, “Categorical reparameterization with Gumbel-Softmax,” in *5th International Conference on Learning Representations (ICLR)*, 2017, arXiv:1611.01144.
- [75] A. Krause, P. Perona, and R. Gomes, “Discriminative clustering by regularized information maximization,” in *Advances in Neural Information Processing Systems*, vol. 23. Curran Associates, Inc., 2010.
- [76] Y. Grandvalet and Y. Bengio, “Semi-supervised learning by entropy minimization,” in *Advances in Neural Information Processing Systems*, vol. 17. MIT Press, 2004.
- [77] G. Pereyra, G. Tucker, J. Chorowski, L. Kaiser, and G. Hinton, “Regularizing neural networks by penalizing confident output distributions,” 2017.
- [78] A. Kumar, A. Raghunathan, R. Jones, T. Ma, and P. Liang, “Fine-tuning can distort pretrained features and underperform out-of-distribution,” in *10th International Conference on Learning Representations (ICLR)*, 2022, arXiv:2202.10054.
- [79] J. Howard and S. Ruder, “Universal language model fine-tuning for text classification,” in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2018, pp. 328–339.
- [80] K. He, R. Girshick, and P. Dollár, “Rethinking ImageNet pre-training,” in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 4917–4926.
- [81] A. Radford, L. Metz, and S. Chintala, “Unsupervised representation learning with deep convolutional generative adversarial networks,” in *4th International Conference on Learning Representations (ICLR)*, 2016, arXiv:1511.06434.
- [82] D. Ulyanov, A. Vedaldi, and V. Lempitsky, “Instance normalization: The missing ingredient for fast stylization,” 2016.
- [83] A. Brock, J. Donahue, and K. Simonyan, “Large scale GAN training for high fidelity natural image synthesis,” in *7th International Conference on Learning Representations (ICLR)*, 2019, arXiv:1809.11096.
- [84] Y. Choi, Y. Uh, J. Yoo, and J.-W. Ha, “StarGAN v2: Diverse image synthesis for multiple domains,” in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 8185–8194.
- [85] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, “Spectral normalization for generative adversarial networks,” in *6th International Conference on Learning Representations (ICLR)*, 2018, arXiv:1802.05957.
- [86] S. Reed, Z. Akata, X. Yan, L. Logeswaran, B. Schiele, and H. Lee, “Generative adversarial text to image synthesis,” in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 48. PMLR, 2016, pp. 1060–1069.
- [87] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *3rd International Conference on Learning Representations (ICLR)*, 2015, arXiv:1412.6980.
- [88] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, “GANs trained by a two time-scale update rule converge to a local Nash equilibrium,” in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017.
- [89] D.-P. Fan, S. Zhang, Y.-H. Wu, Y. Liu, M.-M. Cheng, B. Ren, P. L. Rosin, and R. Ji, “Scoot: A perceptual metric for facial sketches,” in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 5611–5621.
- [90] M. J. Chong and D. Forsyth, “Effectively unbiased FID and inception score and where to find them,” in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 6069–6078.
- [91] G. Parmar, R. Zhang, and J.-Y. Zhu, “On aliased resizing and surprising subtleties in GAN evaluation,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 11 400–11 410.

- [92] M. Bińkowski, D. J. Sutherland, M. Arbel, and A. Gretton, “Demystifying MMD GANs,” in *6th International Conference on Learning Representations (ICLR)*, 2018, arXiv:1801.01401.
- [93] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 586–595.
- [94] A. Borji, “Pros and cons of GAN evaluation measures,” *Computer Vision and Image Understanding*, vol. 179, pp. 41–65, 2019.
- [95] M. Lucic, K. Kurach, M. Michalski, S. Gelly, and O. Bousquet, “Are GANs created equal? a large-scale study,” in *Advances in Neural Information Processing Systems*, vol. 31. Curran Associates, Inc., 2018.
- [96] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32. Curran Associates, Inc., 2019.
- [97] NVIDIA, “NVIDIA T4 tensor core GPU,” <https://www.nvidia.com/en-us/data-center/tesla-t4/>, 2026, accessed 3 October 2026.
- [98] Google, “Google Colaboratory,” <https://colab.research.google.com/>, 2026, accessed 3 October 2026.
- [99] Optuna, “Optuna: A hyperparameter optimization framework — documentation,” <https://optuna.readthedocs.io/en/stable/>, 2026, accessed 3 October 2026.
- [100] S. Watanabe, “Tree-structured Parzen estimator: Understanding its algorithm components and their roles for better empirical performance,” 2023.
- [101] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012.
- [102] K. Jamieson and A. Talwalkar, “Non-stochastic best arm identification and hyperparameter optimization,” in *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 51. PMLR, 2016, pp. 240–248.
- [103] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, “Hyperband: A novel bandit-based approach to hyperparameter optimization,” *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018.
- [104] L. Li, K. Jamieson, A. Rostamizadeh, E. Gonina, M. Hardt, B. Recht, and A. Talwalkar, “A system for massively parallel hyperparameter tuning,” in *Proceedings of Machine Learning and Systems (MLSys)*, 2020, arXiv:1810.05934.
- [105] Weights & Biases, “Weights & Biases: The AI developer platform,” <https://wandb.ai/site/>, 2026, accessed 3 October 2026.
- [106] Google, “Stitch: Design with AI,” <https://stitch.withgoogle.com/>, 2026, accessed 3 October 2026.
- [107] S. Ramírez, “FastAPI documentation,” <https://fastapi.tiangolo.com/>, 2026, accessed 3 October 2026.
- [108] ONNX Runtime, “ONNX Runtime documentation,” <https://onnxruntime.ai/docs/>, 2026, accessed 3 October 2026.
- [109] Meta Open Source, “React: The library for web and native user interfaces,” <https://react.dev/>, 2026, accessed 3 October 2026.
- [110] Tailwind Labs, “Tailwind CSS,” <https://tailwindcss.com/>, 2026, accessed 3 October 2026.
- [111] Docker, Inc., “Docker Compose documentation,” <https://docs.docker.com/compose/>, 2026, accessed 3 October 2026.
- [112] ONNX, “ONNX: Open neural network exchange,” <https://onnx.ai/>, 2026, accessed 3 October 2026.
- [113] PyTorch, “torch.onnx — PyTorch documentation,” <https://docs.pytorch.org/docs/stable/onnx.html>, 2026, accessed 3 October 2026.
- [114] Q. Huynh-Thu and M. Ghanbari, “Scope of validity of PSNR in image/video quality assessment,” *Electronics Letters*, vol. 44, no. 13, pp. 800–801, 2008.